DD

v1

Francesco Gangi, Denis Sanduleanu

December 24, 2024

Contents

1 Introduction

1.1 Purpose

This document contains the design description of the Students&Companies system. It includes the architectural design, the user interface design and the description of all the operations that the system will perform. It also shows how the requirements and use cases detailed in the RASD are satisfied by the design of the system. This document is intended to be read by the developers of the system, the testers and the project managers. It is also intended to be used as a reference for the future maintenance of the system.

1.2 Scope

S&C is a system that helps university students and companies to find a perfect match for potential internship positions. From the student perspective, through this platform it's possible to search for internships, apply for them and go through interviews directly with companies in order to start an internship. On the other hand, from companies perspective, it's possible to publish new internship opportunities, find suitable profiles for your published internships and have a direct contact with interested students. A more detailed description of the system can be found in the RASD, while the aim of this document is to explain thoroughly how to build the actual system in order to fulfill the requirements and use cases mentioned in the other document. It'll describe the actual structure of the system and give a detailed view of how all the components work together.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Acronyms

- **S&C**: Students&Companies
- **HR**: Human Resources
- **CV**: Curriculum Vitae

1.4 Revision History

- **Version 1.0** (Dec. 24, 2024)

1.5 Reference Documents

- **Assignment Document** (Assignment RASD&DD A.Y. 2024-2025)
- **Course Slides** (Software Engineering 2 A.Y. 2024-2025)

1.6 Document Structure

1. **Introduction**: it aims to give a brief recap of the description of the project explained in detail in the RASD.

2. **Architectural Design**: this section provides a description of the architecture of the system, including the components and the interfaces between them. It also includes the runtime view of the most important operations of the system, the deployment view and the architectural styles and patterns used in the system.

3. **User Interface Design**: it includes the mock-ups of the application user interfaces, already mentioned in the RASD.

4. **Requirements Traceability**: it describes the connections between the requirements defined in the RASD and the components described in the first chapter. This is used as proof that the design decisions have been taken with respect to the requirements, and therefore that the designed system can fulfill the goals.

5. **Implementation, Integration and Test Plan**: it describes the process of implementation, integration and testing to which developers have to stick in order to develop the system in a correct way.

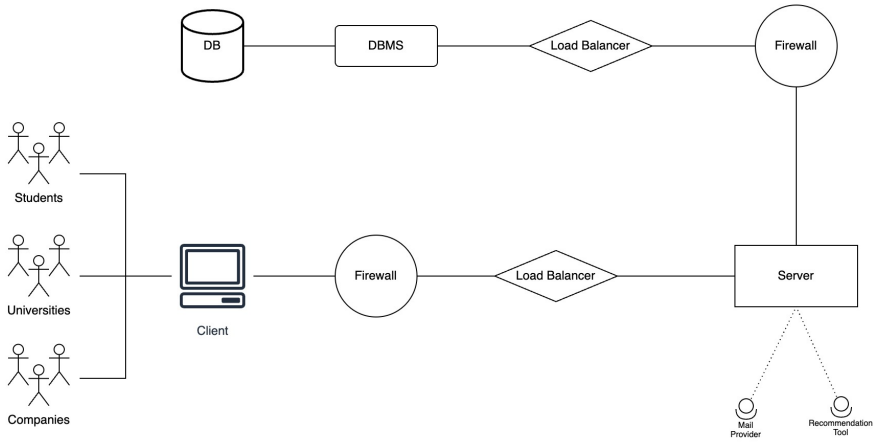
6. **Effort spent**: it shows the time spent to realize this document organized by sections.

7. **References**: it contains the references to any documents and software used in this document.

2 Architectural Design

2.1 Overview

In Students&Companies architecture we will take advantage of a canonical three-tier architecture composed by the following layers: **Client**, **Server** and **DBMS**. The **Client** (*Presentation Layer*) is the one that interacts with the user, showing the user interface and receiving the user inputs. The **Server** (*Application Layer*) is the one that manages the business logic of the system, it receives the inputs from the Client, processes them and sends the results to the DBMS. The **DBMS** (*Data Layer*) is the one that manages the data of the system, it receives the inputs from the Application Layer, processes them and sends the results back to the Application Layer. The DBMS is also responsible for the communication with the database.



High-level view of the Architecture

v

2.2 Component view

Here are listed the components described in the following view with their own use and description.

Application Layer

- **Dashboard Manager**: it provides the functionalities to interact with a personal dashboard by a user.
- **Authentication Service**: it provides the procedures that make the authentication of a given user possible.
- **Notification Service**: it handles the interactions with a mail provider and how to notify users within the application.
- **Internship Manager**: it handles the internships by a given user and interacts with the ***Application Manager*** to manage every ongoing internship singularly.
- **Application Manager**: it allows to individually monitor the progress of an ongoing internship and deal with all of the potential problems or complaints within it.
- **Data Manager**: it's the crucial component needed for data exchange between ***Data Layer*** and ***Application Layer*** and it basically interacts with every other component within the ***Server***.

Presentation Layer

- **Client**: it represents the front-end side of the application and can be accessed through a generic web browser. Through the ***Authentication Interface*** it's possible to access to a given user page and all of the different operations are possible due to the ***Dashboard Interface***.

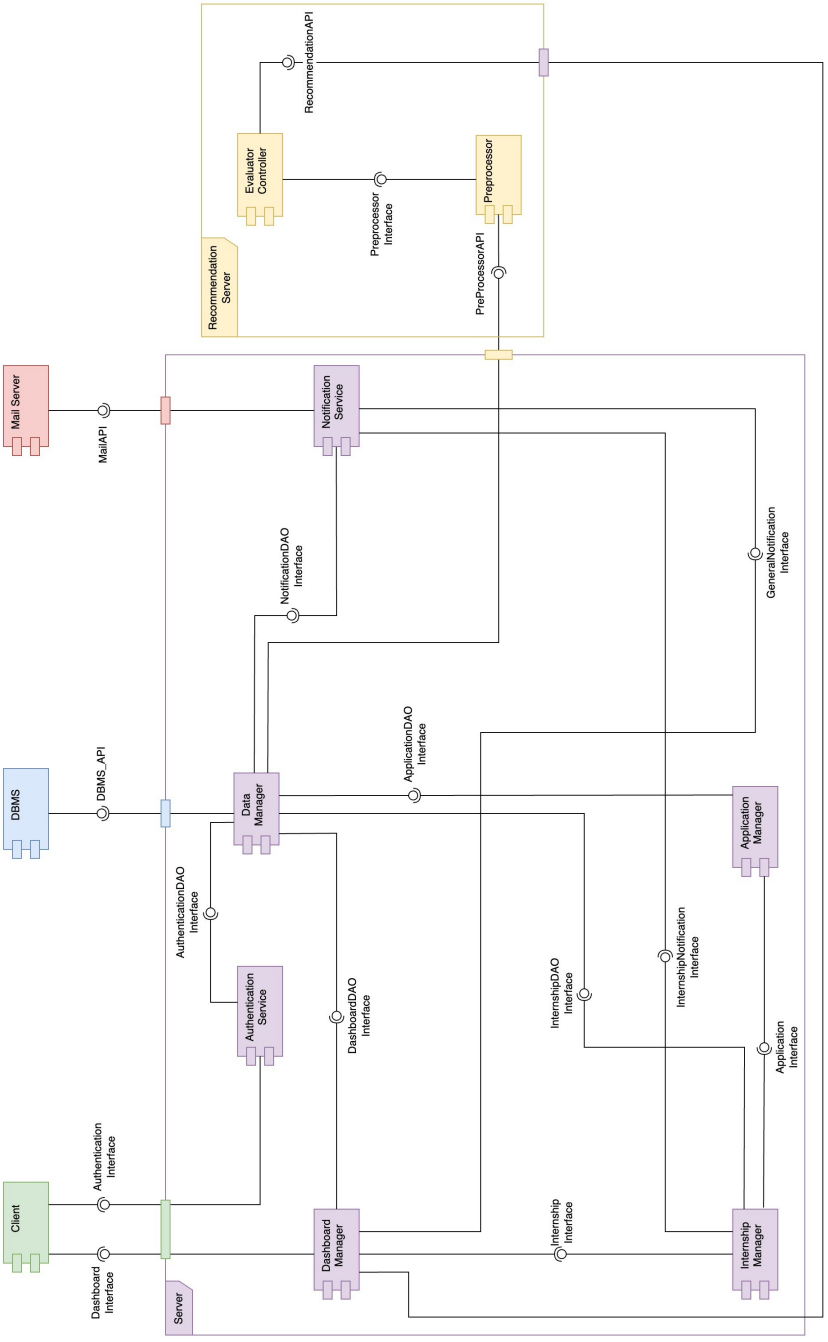
Data Layer

- **DBMS**: the component needed to access and manages transactions between the database and the other layers of the three-tier architecture.

Other services

- **Mail Server**: it's needed to exchange emails between the application actors.

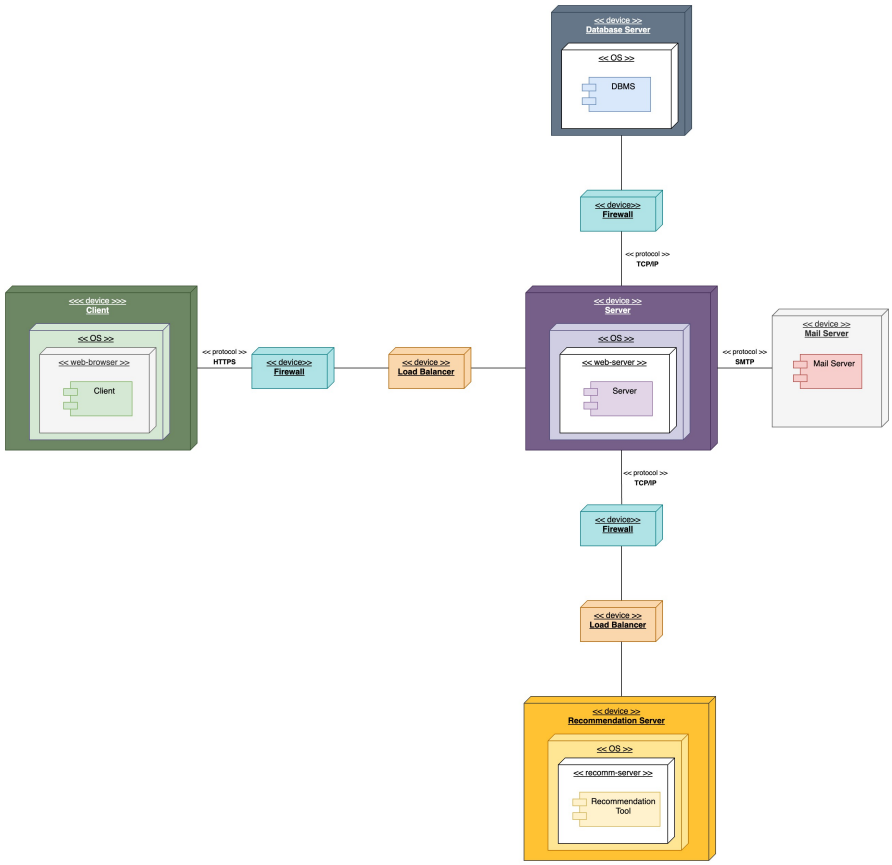
- **Recommendation Server**: the tool that is in charge of recommending suitable internships to students, as well as suitable students to companies. It consists of a ***Preprocessor*** that manages all of the data and a ***Evaluator Controller*** that actually takes account of the ratings and other statistics within the application in order to generate a proper recommendation.



Component view

2.3 Deployment view

In this section it's shown the deployment of the system. The overall architecture, including the Recommendation Tool, becomes a four-tier architecture. The **Client** layer represents the web browser through which it's possible to access the application for a user. The **Server** is the layer that keeps all of the information about the application and manages all the interactions with other components such as external services or the database.



Deployment view

2.3.1 Firewall

In order to protect the application from external attacks or vulnerabilities, the best approach was to use firewalls to filter both incoming and outgoing traffic. It was chosen to use one between ***Client*** and ***Server***, one between ***Server*** and ***Recommendation Server*** and the last one between ***Server*** and ***Database Server***.

2.3.2 Load Balancing

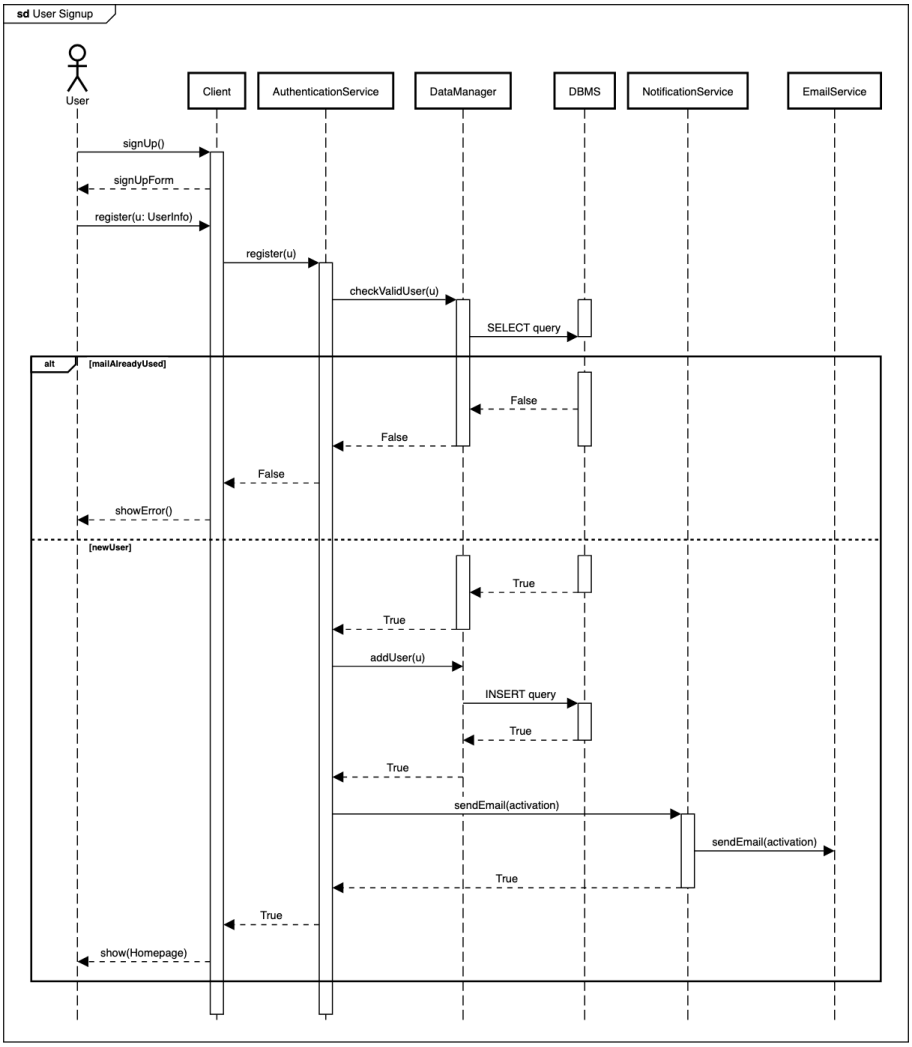
Since the application is going to run on replicated servers we want to ensure that users are equally partitioned between different replicas. In order to ensure that, we used two load balancers: one of them can be found between ***Client*** and ***Server***, while the other one is located between ***Server*** and ***Recommendation Server***.

2.4 Runtime view

In this section are shown the sequence diagrams of the most important operations of the system. The diagrams show the interactions between the components of the system and how they work together to achieve the desired result.

User Sign Up

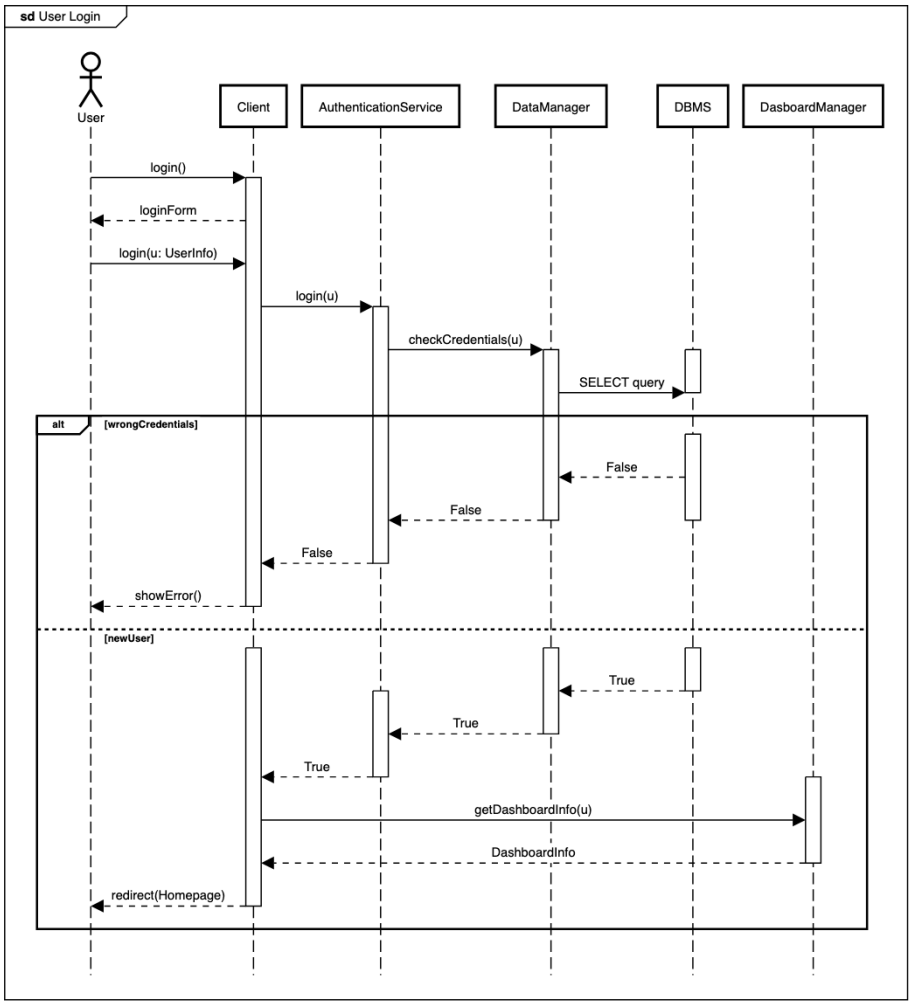
When a user wants to sign up to the application, the ***Client*** sends a request to the ***Authentication Service***. The ***Authentication Service*** checks if the user is already registered, if not it sends the user information to the ***Data Manager*** that stores the information in the database. The system checks for activation and sends an email to the user to activate the account. Finally, it returns the homepage to the user.



User Sign Up

User Login

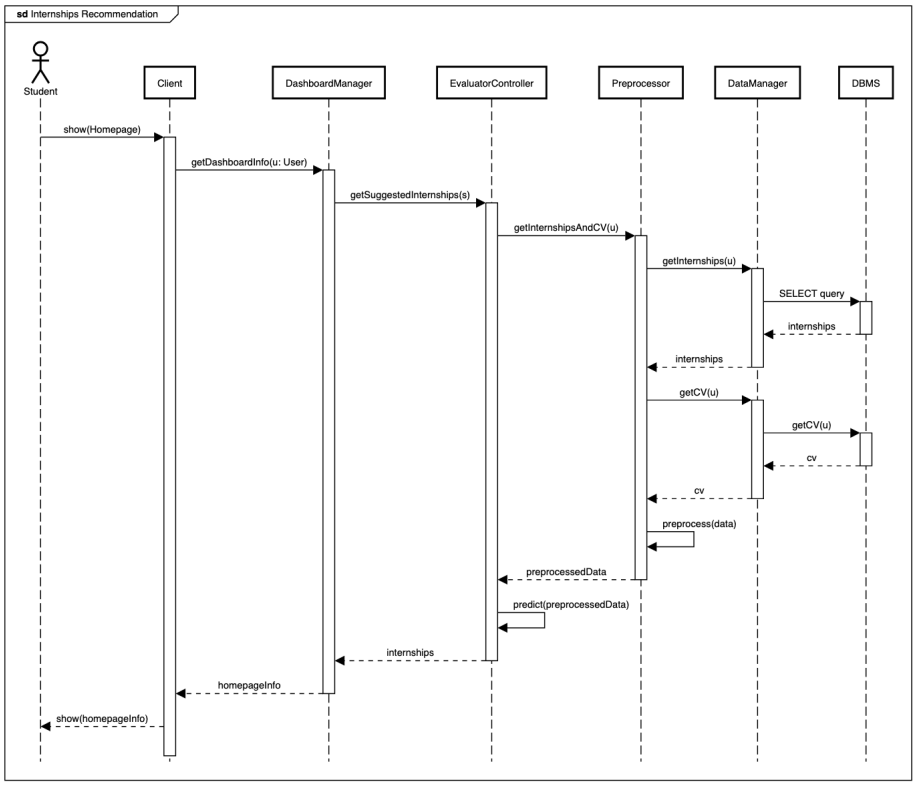
When a user wants to log in to the application, the ***Client*** sends a request to the ***Authentication Service***. The ***Authentication Service*** checks if the user is registered and if the credentials are correct. If the credentials are correct, the system returns the homepage to the user.



User Login

Internships Recommendation

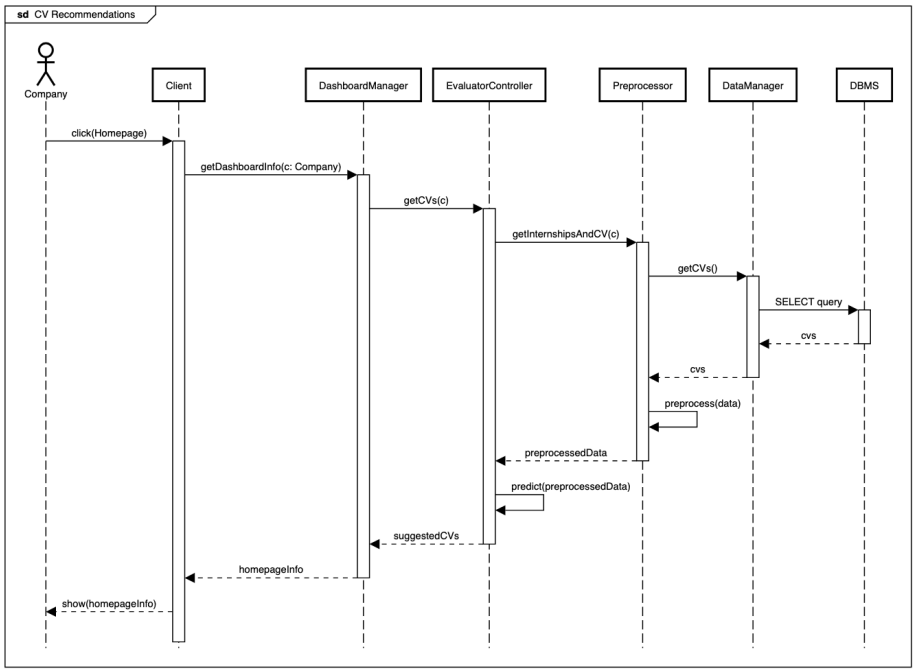
When a Student enters on S&C, the ***Dashboard Manager*** requests to the ***Evaluator Controller*** the recommended Internships to be shown to the Student. The **Preprocessor** is used to pre-process the data in order to be correctly fed to the machine learning algorithms of the Evaluator Controller.



Internships Recommendation

Students Recommendation

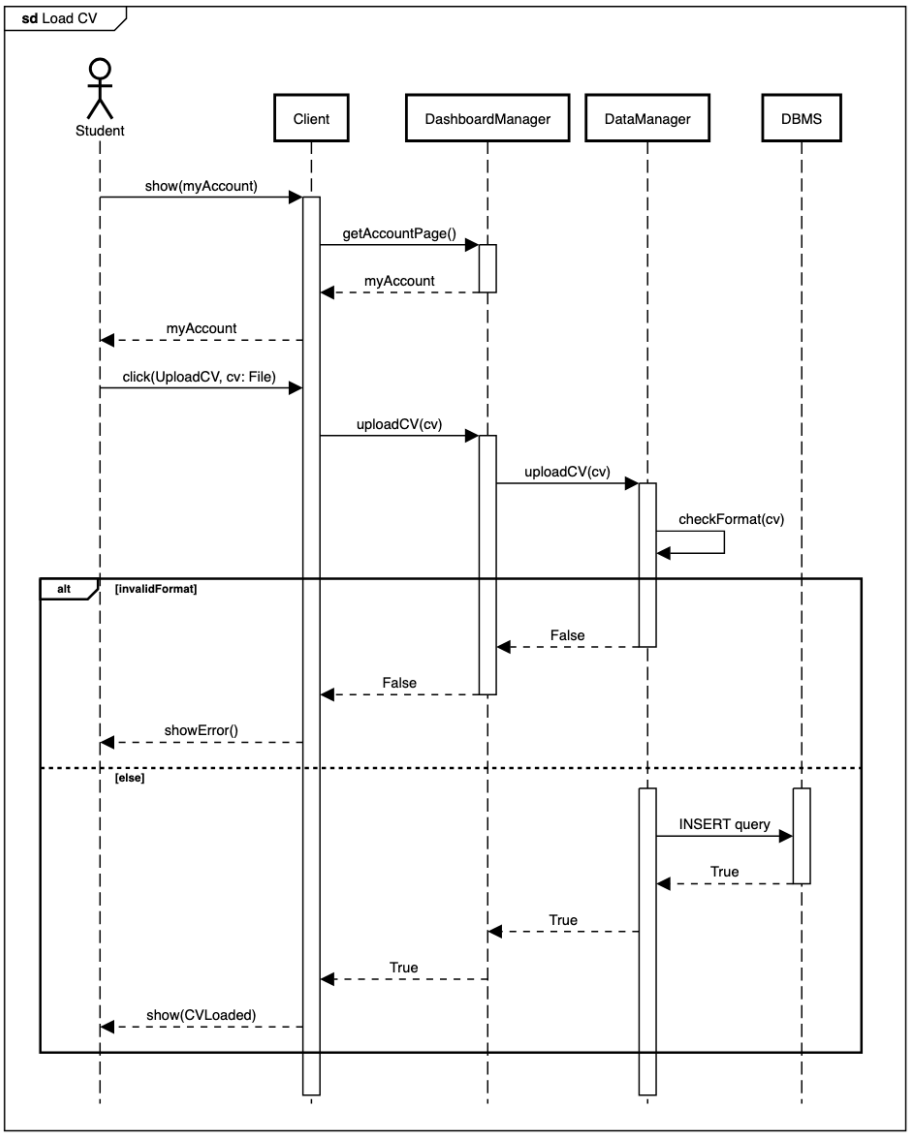
Similarly to what is done in Internship Recommendation, when a Company logs into S&C, machine learning algorithms from the ***Evaluator Controller*** are used in order to recommend interesting Students for their Internships.



Students Recommendation

Load CV

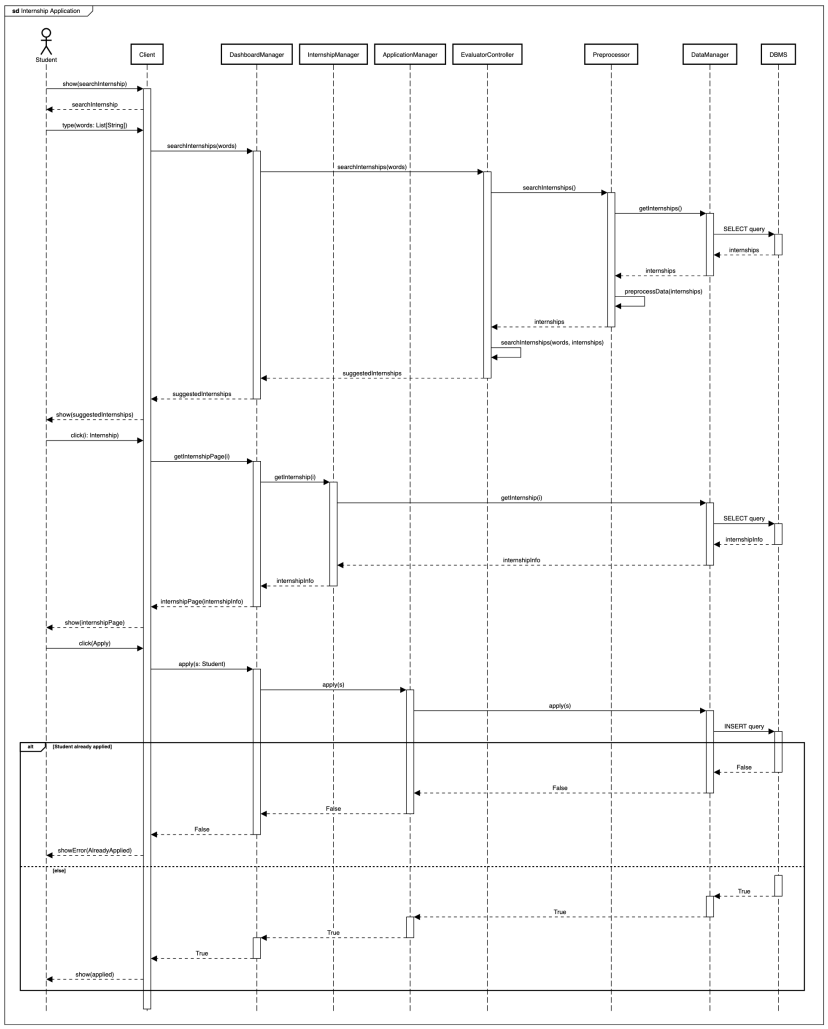
When a student wants to load his CV, the ***Client*** sends a request to the ***Data Manager***. The system checks if the CV file format is correct and stores the CV in the database. Finally, it returns the loaded CV to the student.



Load CV

Internship Application

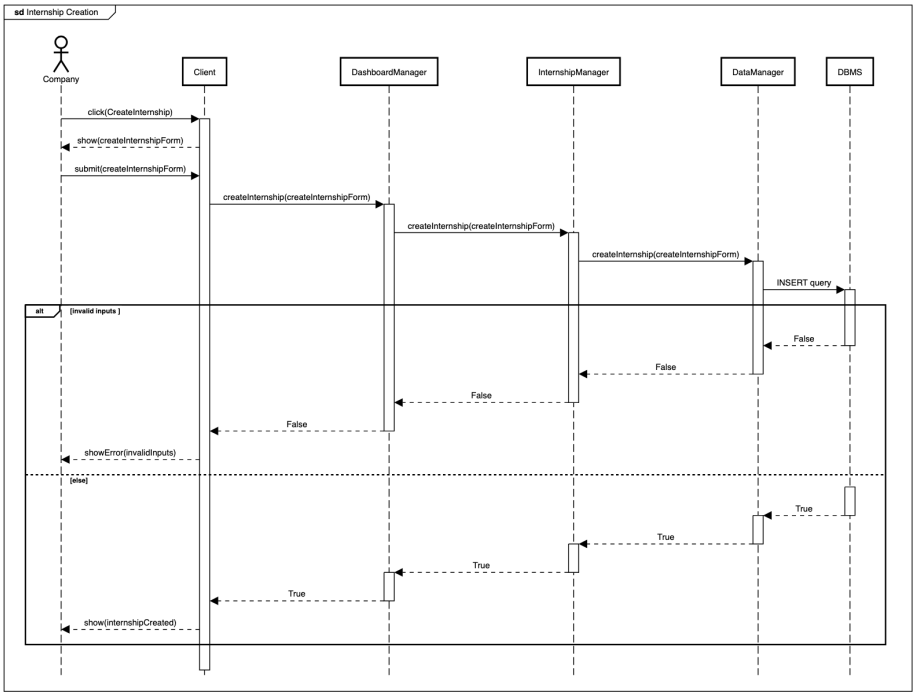
When a student wants to apply for an internship, the ***Client*** sends a request to the ***Dashboard Manager*** by searching for internships. The request goes through the ***Evaluator Controller*** that provides the student with a list of suggested internships. The student selects an internship and sends the application to the ***Application Manager***. The system checks if the student has already applied for that internship and if not, it stores the application in the database. Finally, it returns the application status to the student.



Internship Application

Internship Creation

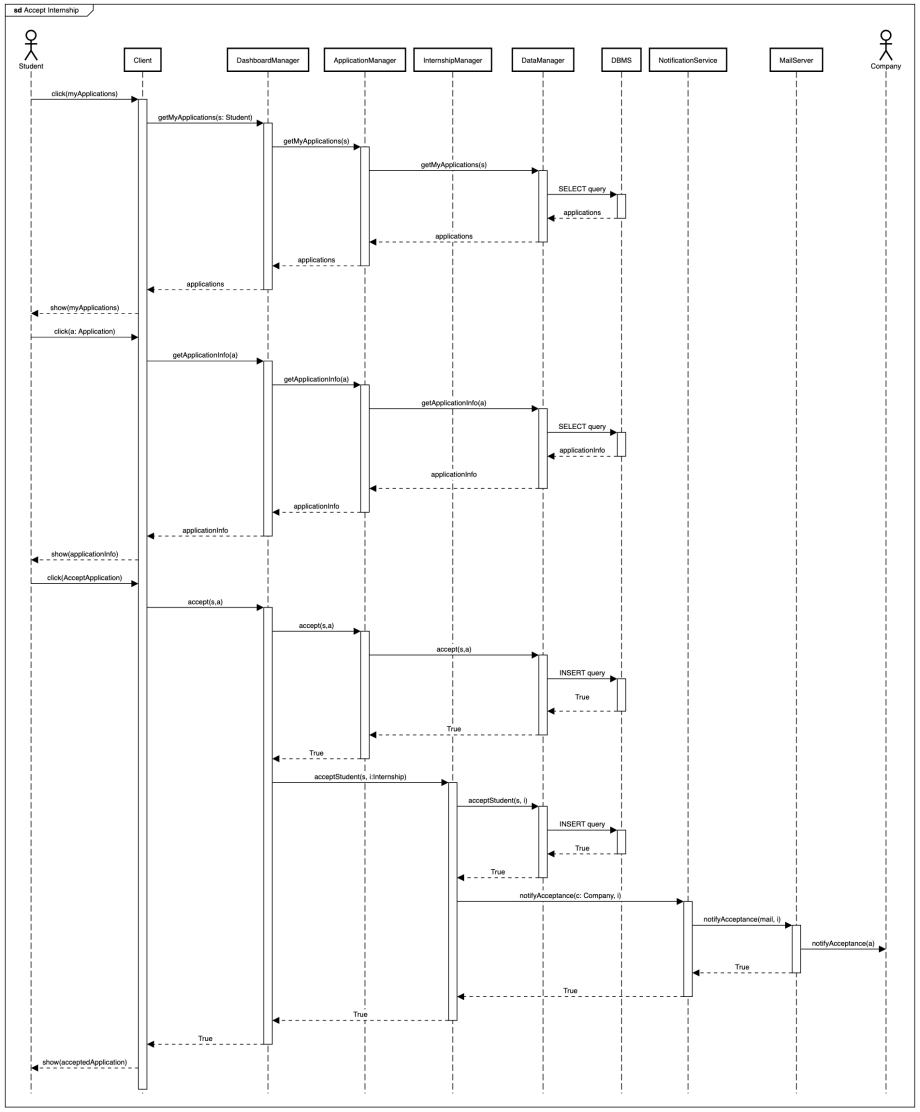
When a company wants to create an internship, the ***Client*** sends a request to the ***Dashboard Manager***. The request is then sent to the ***Internship Manager*** that stores the internship in the database. The system checks if the internship has been created successfully and returns the internship status to the company.



Internship Creation

Accept Internship

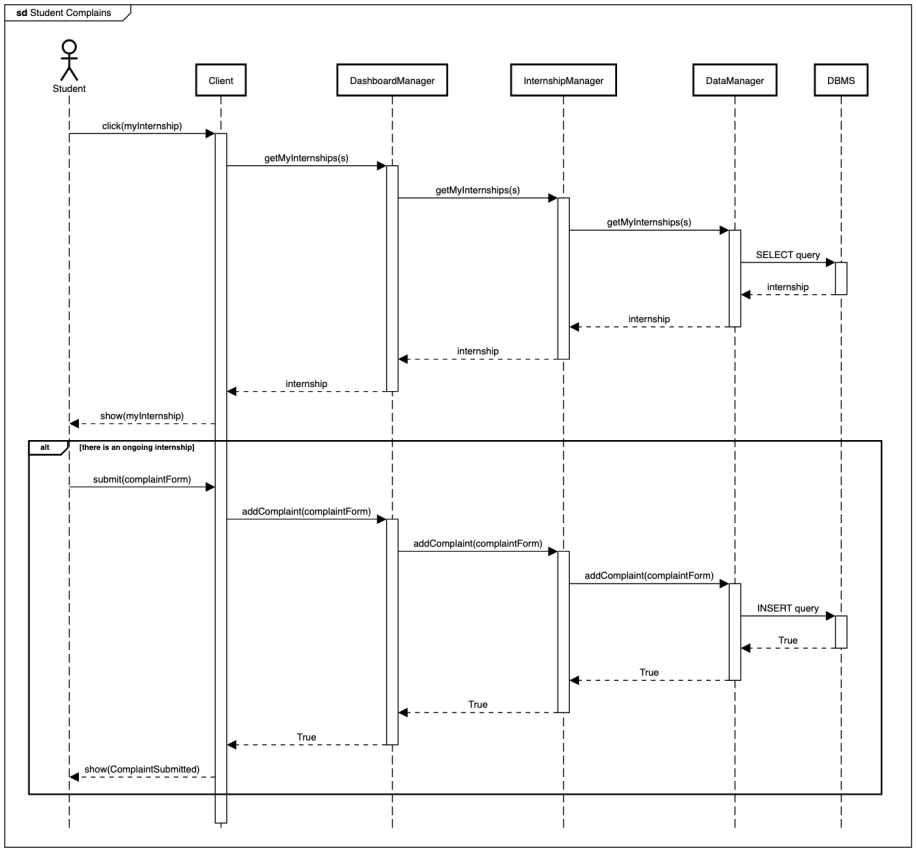
When a student wants to accept an internship from the ones they applied for, the ***Client*** sends a request to the ***Dashboard Manager***. The request is then sent to the ***Application Manager*** that stores the selection in the database. The ***Notification Service*** sends an email to the company to notify them about the student's acceptance. Finally, the system returns the internship status to the student.



Accept Internship

Student Complaint

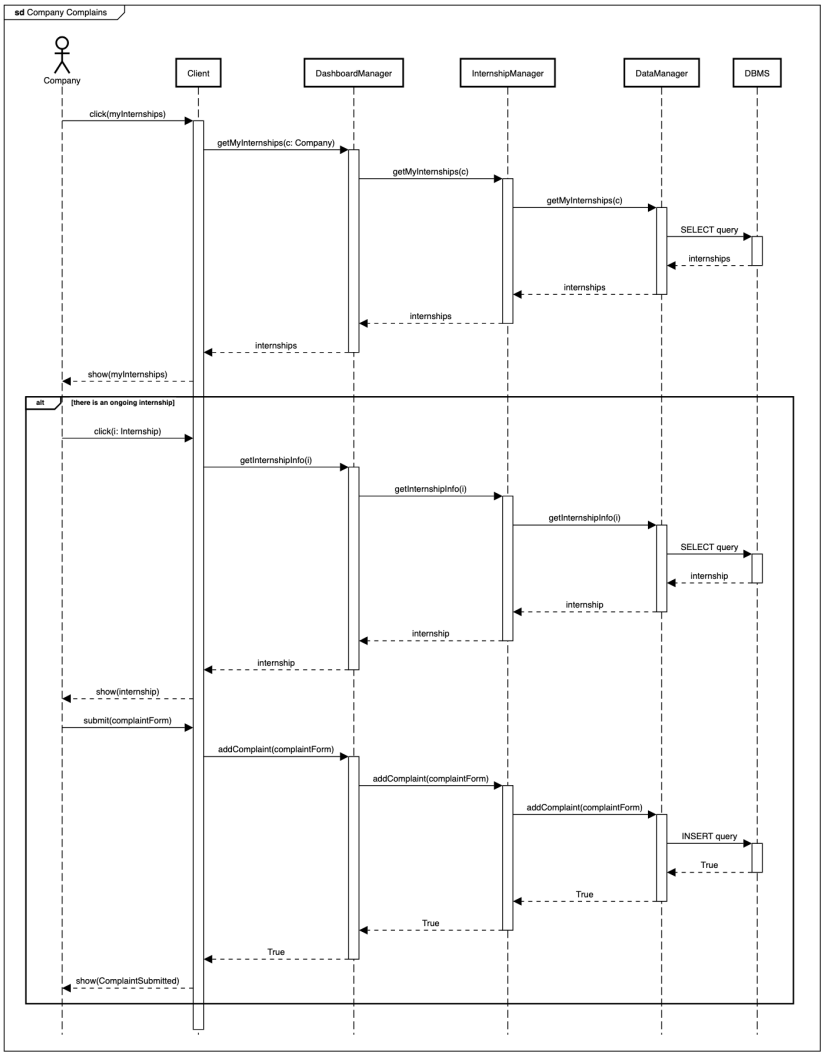
When a student wants to complain about an internship, the ***Client*** sends a request to the ***Dashboard Manager***. The request is then sent to the ***Internship Manager*** that checks if there's an ongoing internship and stores the complaint in the database. Finally, the system returns the complaint status to the student.



Student complains during Internship

Company Complaint

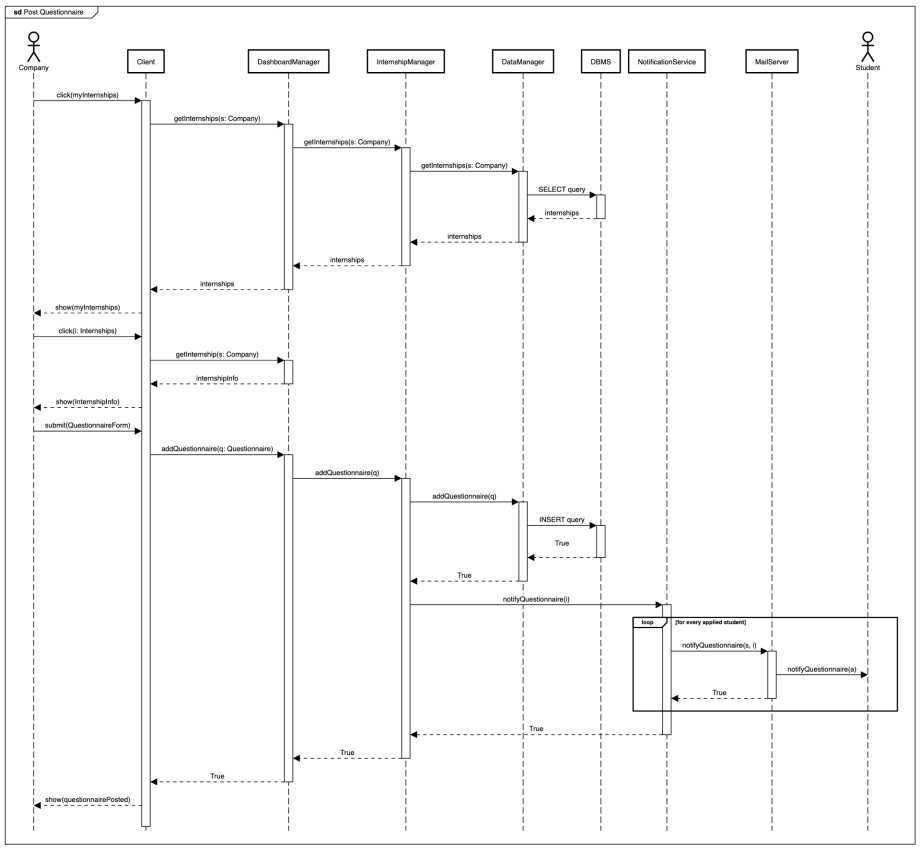
When a company wants to complain about a student, the ***Client*** sends a request to the ***Dashboard Manager***. The request is then sent to the ***Internship Manager*** that checks if there's an ongoing internship and stores the complaint in the database. Finally, the system returns the complaint status to the company.



Company complains during Internship

Questionnaire Creation

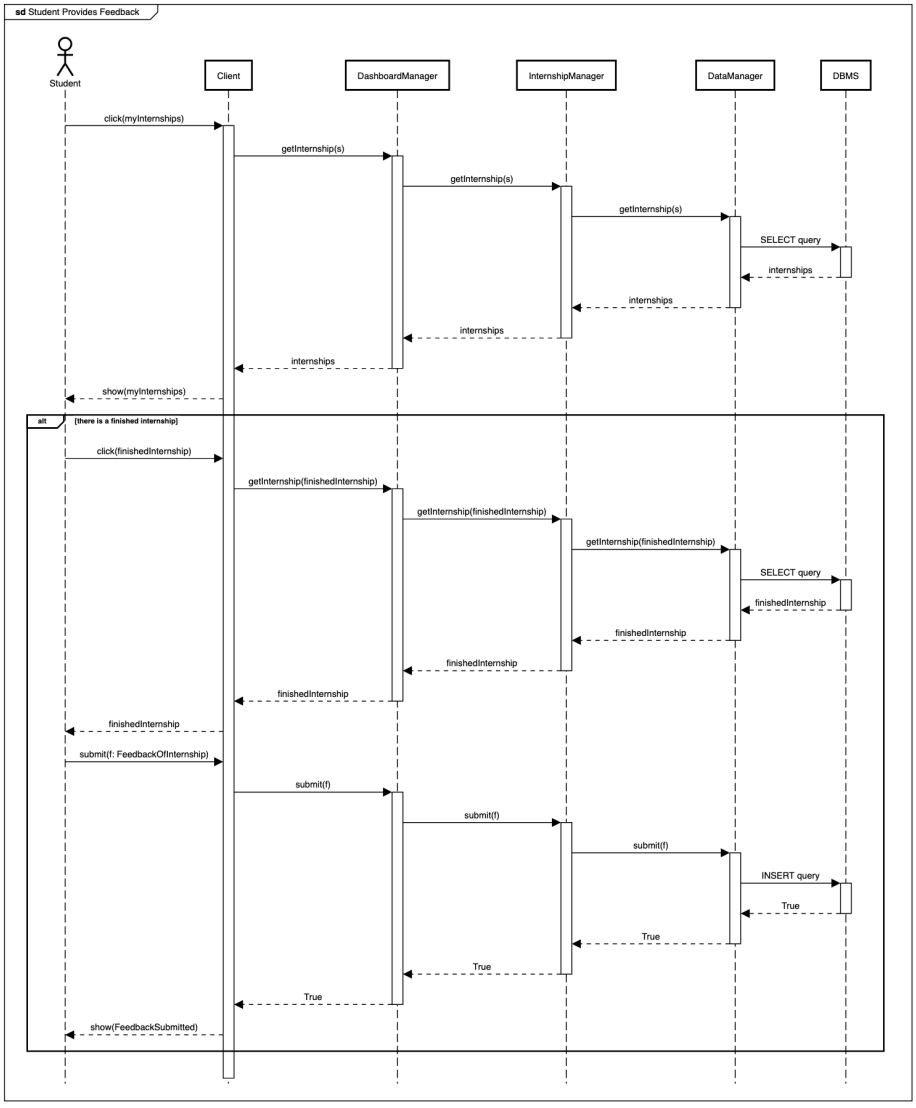
When a company wants to create a questionnaire for an internship, the ***Client*** sends a request to the ***Dashboard Manager***. The request is then sent to the ***Internship Manager*** that stores the questionnaire in the database. After publishing the questionnaire, the ***Notification Service*** sends an email only to students that applied for that internship to notify them about the questionnaire. Finally, the system returns the questionnaire status to the company.



Post Questionnaire

Student's Feedback Submission

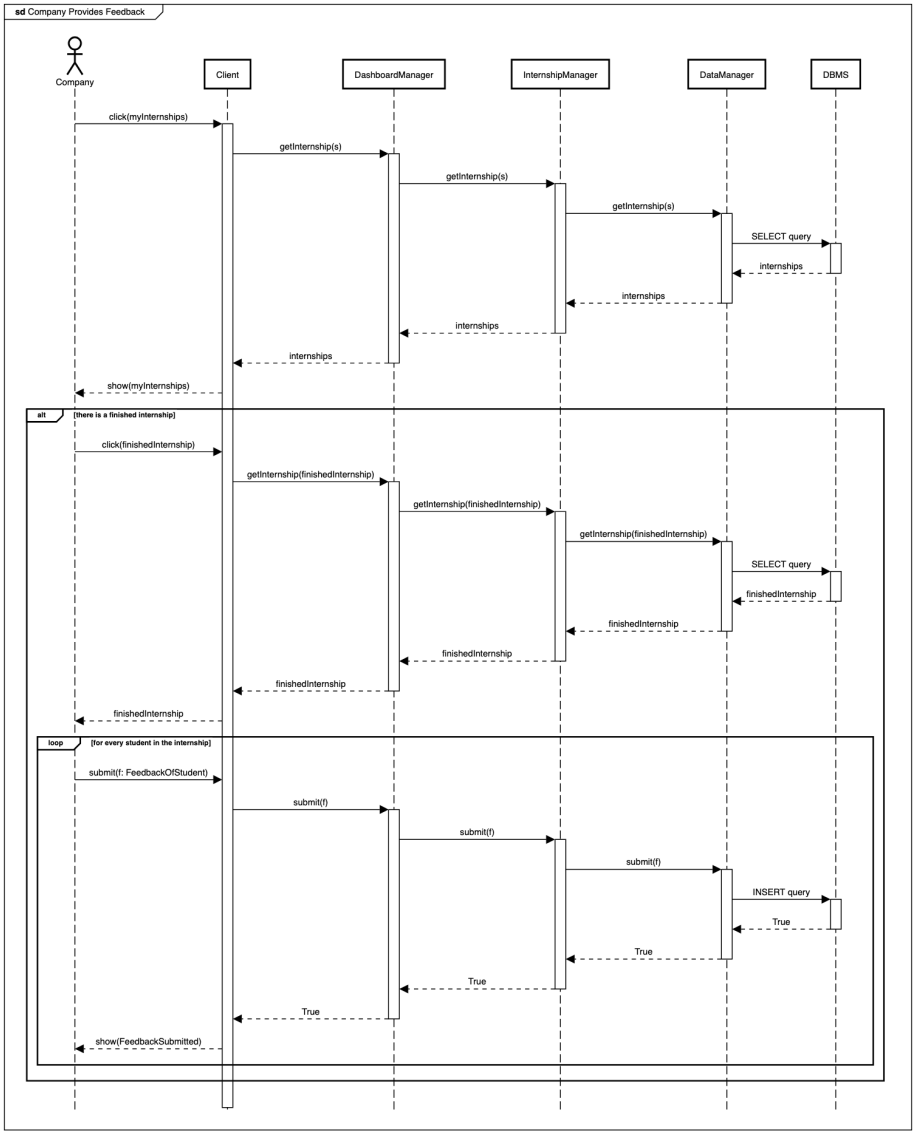
When a student wants to submit feedback about an internship, the ***Client*** sends a request to the ***Dashboard Manager***. The request is then sent to the ***Internship Manager*** that retrieves the internships the student has been part of and the student selects the internship to provide feedback for. The system stores the feedback in the database and returns the feedback status to the student.



Student provides Feedback

Company's Feedback Submission

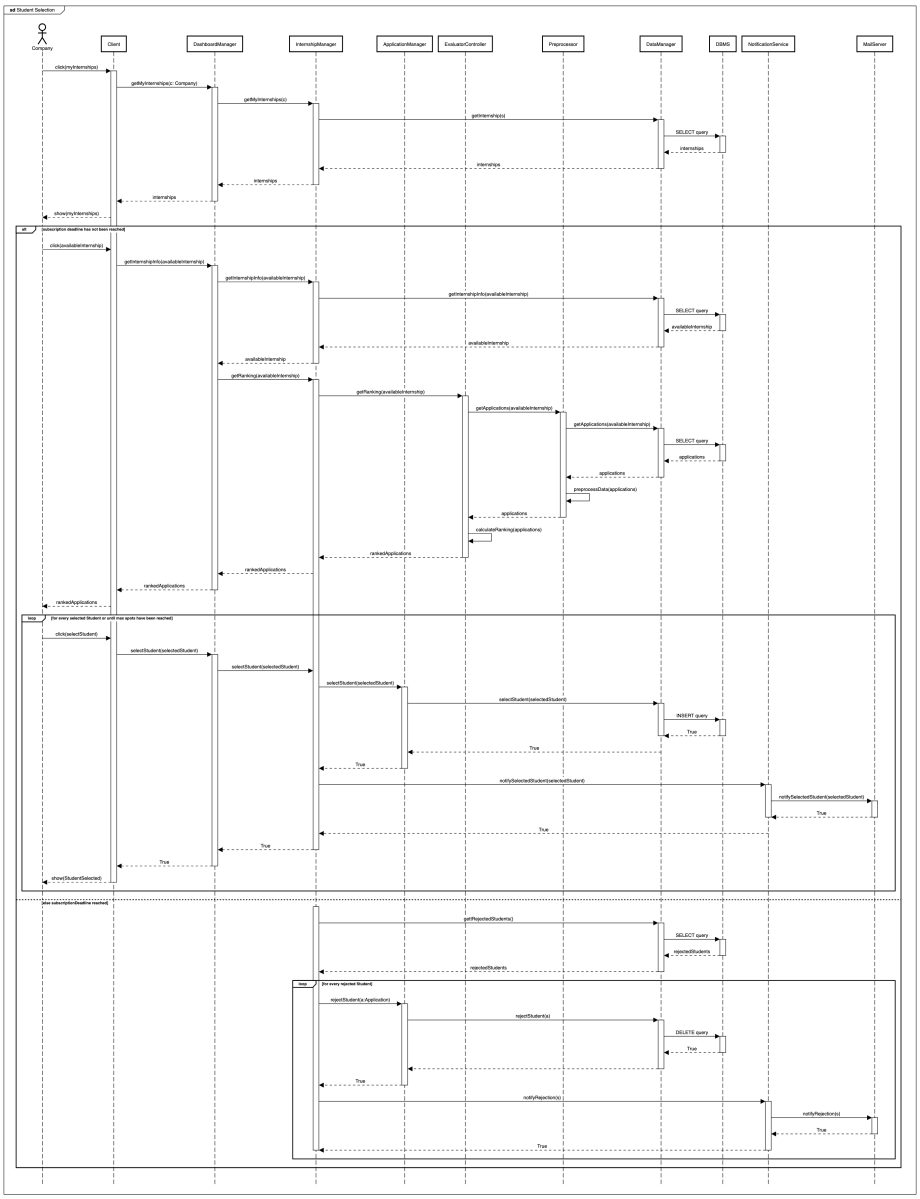
When a company wants to submit feedback about a student, the ***Client*** sends a request to the ***Dashboard Manager***. The request is then sent to the ***Internship Manager*** that retrieves the internships the company has created selecting the one that has finished. The company provides feedback about every student that has been part of the internship and the system stores the feedback in the database. Finally, the system returns the feedback status to the company.



Company provides Feedback

Student Selection

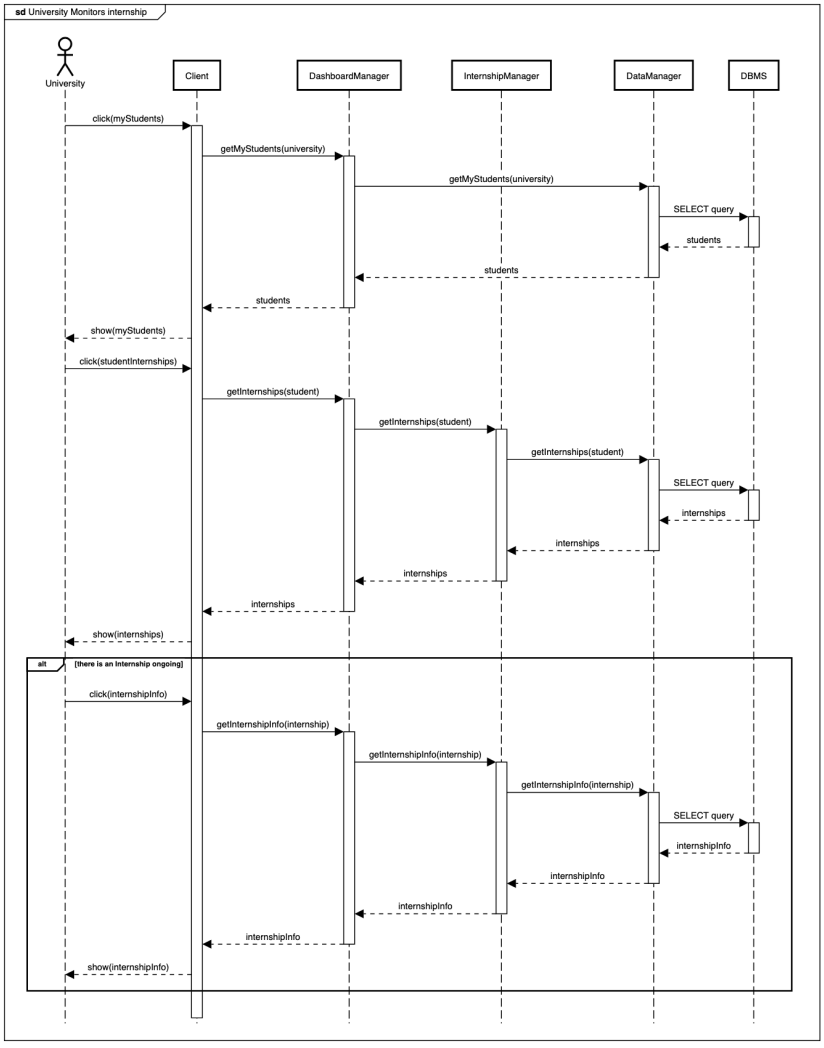
When a company wants to select a student for an internship, the ***Client*** sends a request to the ***Dashboard Manager***. The request is then sent to the ***Internship Manager*** that retrieves the internships the company has created and returns the internships to the company. If a given internship has not exceeded its deadline, the company ranks the students that applied for that internship and selects them as long as there are available positions. The system stores the selection in the database and returns the selection status to the company, while the ***Notification Service*** sends out notifications to inform accepted students about the outcome of the selection process. Otherwise, if an internship is already expired, the system retrieves the list of rejected students and notifies every of them about rejection.



Student Selection

University Monitors Internship

When a university wants to monitor an internship, the ***Client*** sends a request to the textbf*Dashboard Manager*. The request is then sent to the ***Internship Manager*** that retrieves the students that have been part of an internship and selects a student to monitor. The system checks if there's an ongoing internship and retrieves information about a given student with an ongoing internship. Finally, the system returns the student's internship progress to the university.



University monitors internship

2.5 Component interfaces

2.5.1 Authentication Service

Authentication Interface

- `bool register(u:UserInfo)`
- `bool login(email:String, password:  String)`

2.5.2 Dashboard Manager

Dashboard Interface

- `DashboardInfo getDashboardInfo(u:User)`
- `List[Internship] getMyInternships(c:Company)`
- `List[Internship] getMyInternships(s:Student)`
- `Internship getInternshipPage(internship:String)`
- `Internship getInternshipInfo(admin:Company, internship:String)`
- `bool addQuestionnaire(q:Questionnaire)`
- `bool selectStudent(student:String)`
- `List[Application] getMyApplications(s:Student)`
- `Application getApplicationInfo(application:String)`
- `bool accept(s:Student, a:Application)`
- `bool addComplaint(c:Company, form:ComplaintAboutStudent)`
- `bool submit(c:Company, form:FeedbackOfStudent)`
- `bool submit(s:Student, form:FeedbackOfInternship)`
- `List[Internship] searchInternships(name:String, keywords:List[String])`
- `bool apply(s:Student, i:Internship)`
- `bool createInternship(internshipForm:List[String])`
- `bool addComplaint(s:Student,c:ComplaintAboutInternship)`

2.5.3 Internship Manager

Internship Interface

- `List[Internship] getMyInternships(c:Company)`
- `List[Internship] getMyInternships(s:Student)`
- `Internship getInternshipPage(internship:String)`
- `Internship getInternshipInfo(admin:Company, internship:String)`
- `List[Application] getRanking(availableInternship:Internship)`
- `bool selectStudent(s:Student, i:Internship)`
- `bool submit(c:Company, form:FeedbackOfStudent)`
- `bool submit(s:Student, form:FeedbackOfInternship)`
- `bool addComplaint(c:Company, form:ComplaintAboutStudent)`
- `bool addComplaint(s:Student, c:ComplaintAboutInternship)`
- `bool addQuestionnaire(q:Questionnaire)`
- `bool createInternship(internshipForm:List[String])`
- `bool acceptStudent(s:Student, i:Internship)`
- `bool notifyQuestionnaire(i:Internship)`

2.5.4 Application Manager

Application Interface

- `bool selectStudent(a:Application)`
- `bool rejectStudent(a:Application)`
- `bool apply(s:Student,i:Internship)`
- `List[Application] getMyApplications(s:Student)`
- `Application getApplicationInfo(application:String)`

2.5.5 Notification Service

GeneralNotification Interface

- `bool sendActivation(u:User)`

InternshipNotification Interface

- `bool notifySelectedStudent(s:Student, i:Internship)`
- `bool notifyRejectedStudent(s:Student, i:Internship)`
- `bool notifyQuestionnaire(i:Internship)`
- `bool notifyAcceptance(c:Company, i:Internship)`

2.5.6 Preprocessor

PreprocessorAPI

- `data:PreprocessedData getApplications(i:Internship)`
- `data:PreprocessedData getInternshipsAndCV(s:Student)`
- `data:PreprocessedData searchInternships(s:Student)`
- `data:PreprocessedData getInternshipsAndCVs(c:Company)`

2.5.7 Evaluator Controller

RecommendationAPI

- `List[Internship] getSuggestedInternships(s:Student)`
- `List[Internship] searchInternships(s:Student)`
- `List[CV] getCVs(c:Company)`

2.5.8 Data Manager

AuthenticationDAO Interface

- `bool checkValidUser(u:User)`
- `bool addUser(u:User)`
- `bool checkCredentials(u:User)`

DashboardDAO Interface

- `List[Student] getMyStudents(u:University)`

InternshipDAO Interface

- `List[Internship] getInternships(s:Student)`
- `Internship getInternshipInfo(internship:String)`
- `List[Student] getRejectedStudents(internship:Internship)`
- `bool submit(c:Company, form:FeedbackOfStudent)`
- `bool submit(s:Student, form:FeedbackOfInternship)`
- `List[Internship] getMyInternships(c:Company)`
- `List[Internship] getMyInternships(s:Student)`
- `bool addComplaint(c:Company, form:ComplaintAboutStudent)`
- `bool addComplaint(s:Student, c:ComplaintAboutInternship)`
- `bool addQuestionnaire(q:Questionnaire)`
- `bool createInternship(internshipForm:List[String])`
- `bool acceptStudent(s:Student, i:Internship)`

ApplicationDAO Interface

- `bool selectStudent(s:Student, a:Application)`
- `bool rejectStudent(s:Student, a:Application)`
- `bool apply(s:Student,i:Internship)`
- `List[Application] getMyApplications(s:Student)`
- `Application getApplicationInfo(application:String)`

PreprocessorDAO Interface

- `List[Application] getApplications(i:Internship)`
- `List[Internship] getInternships(s:Student)`
- `List[Internship] getInternships()`
- `CV getCV(s:Student)`
- `List[CV] getCVs()`

NotificationDAO Interface

- `String getUserEmail(userID:int)`
- `List[String] getAppliedStudentsEmails(userIDs:List[int])`
- `String getSelectedStudentEmail(studentID:int)`

2.6 Selected architectural styles and patterns

2.6.1 Client-Server Architecture

The system is based on a client-server architecture. The client is the front-end side of the application and can be accessed through a generic web browser. The server is the back-end side of the application and manages all the business logic of the system. The client and the server communicate through the internet using the HTTP protocol. The overall architecture is a three-tier architecture composed by the Client, the Server and the DBMS.

2.6.2 RESTful API

The system uses a RESTful API to communicate between the Client and the Server. The API is stateless and uses the HTTP protocol to perform the operations. The API is used to perform all the operations of the system, such as user registration, user login, internship creation, internship application, internship selection, student feedback submission, company feedback submission, student selection, university internship monitoring, etc.

2.6.3 Model-View-Controller (MVC) Pattern

The system uses the Model-View-Controller (MVC) pattern to separate the business logic from the user interface. The Model represents the data of the system, the View represents the user interface of the system and the Controller represents the business logic of the system. The MVC pattern is used to make the system more modular and easier to maintain.

2.7 Other design decisions

2.7.1 Database Management System

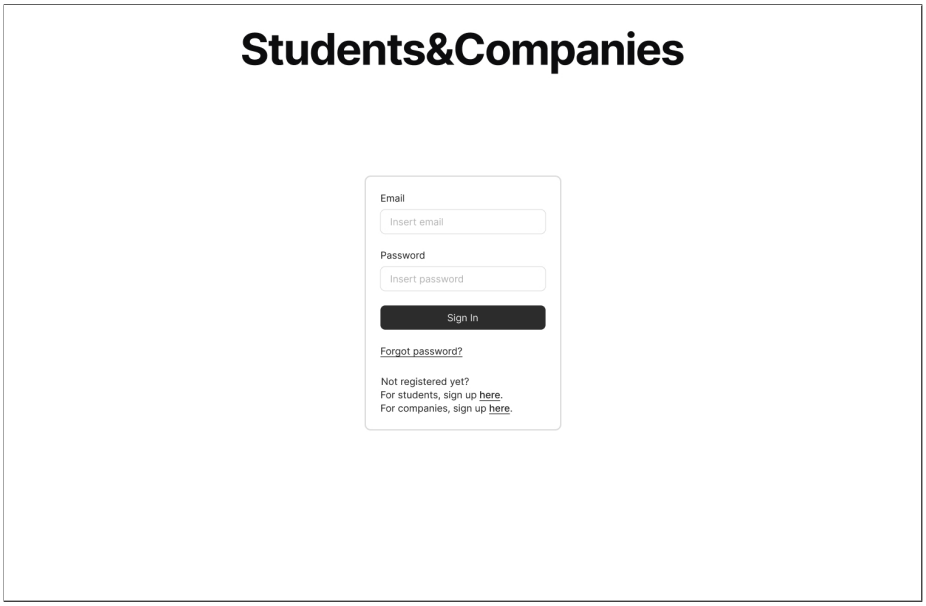
The system uses a Database Management System (DBMS) to store all the data of the system. The DBMS is used to store the user data, the internship data, the application data, the feedback data, the complaint data, the questionnaire data, etc. The DBMS is used to make the data of the system persistent and to make the data of the system easily accessible. Moreover, we chose to implement a relational database to store the data of the system, because of its complex structure and relationships between the entities.

3 User Interface Design

In this section are shown the mock-ups of the application user interfaces. The mock-ups are used to give an idea of how the application will look like and how the user will interact with it. The mock-ups are designed to be simple and intuitive, so that the user can easily understand how to use the application.

Login Page

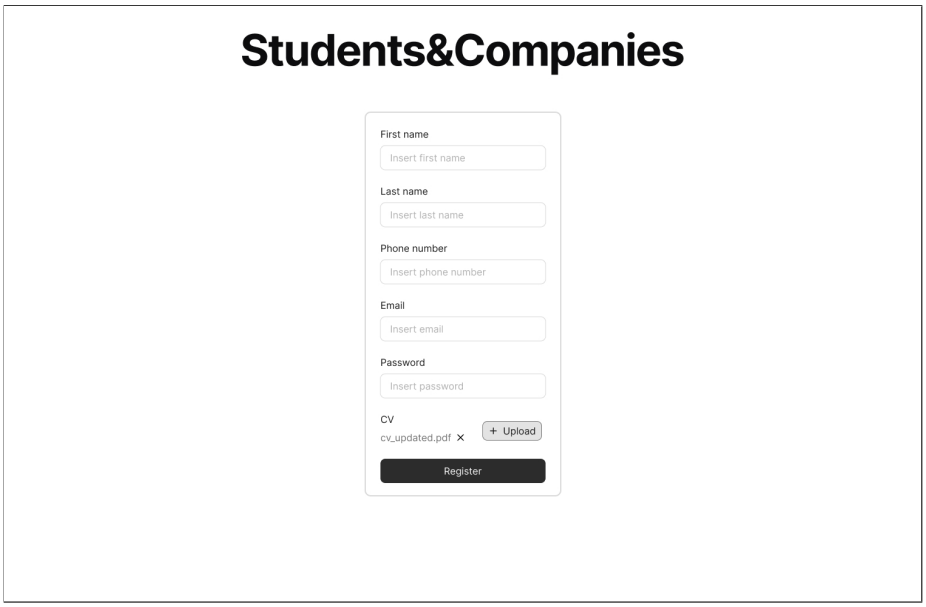
The ***Login Page*** is the first page that the user sees when accessing the application. The user can log in by entering the email and the password. If the user is not registered, the user can sign up by clicking on the "Sign Up" button differentiated for students and companies.



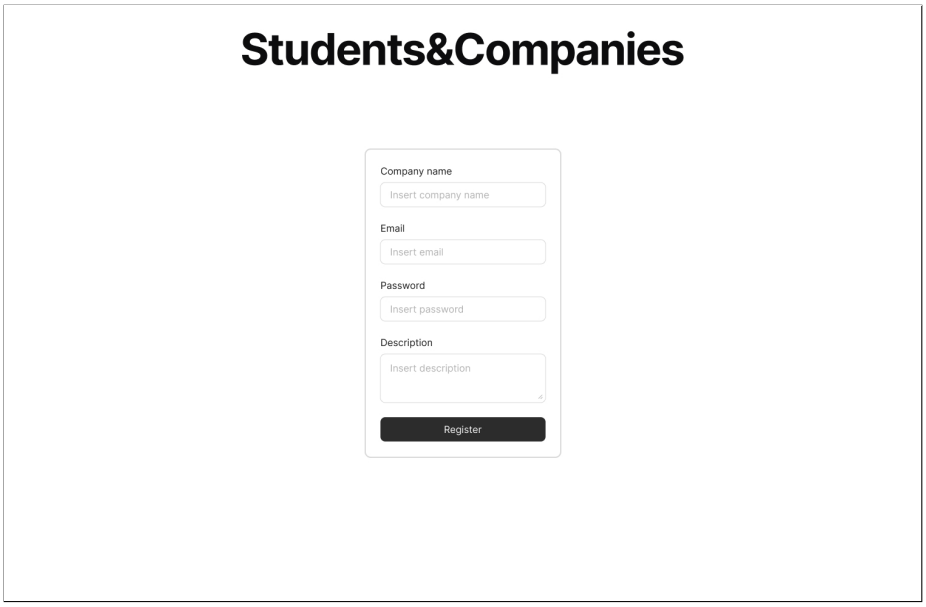
Login Page

Sign Up Page

The ***Sign Up Page*** is the page where the user can register to the application. The user can sign up as a student or as a company by entering the required information.



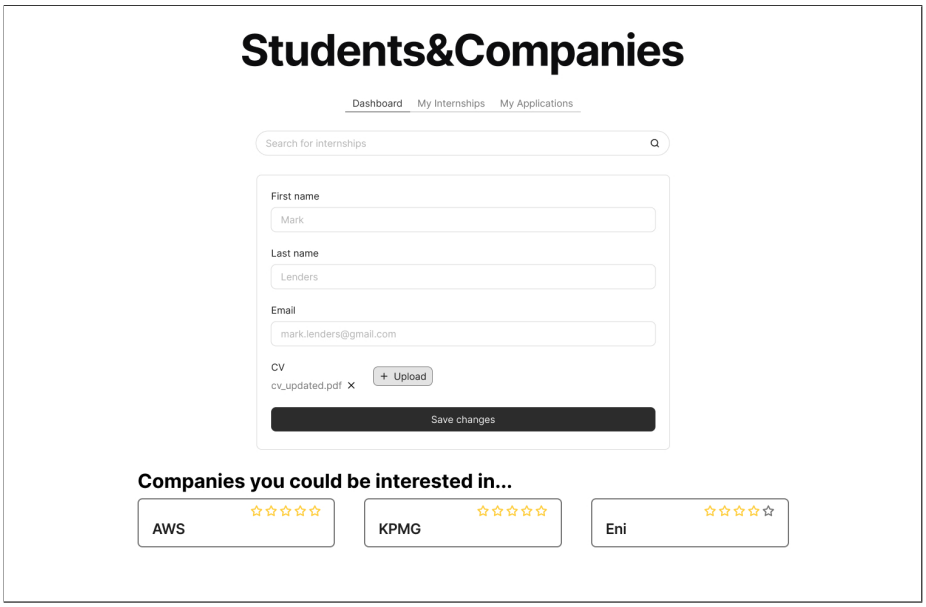
Sign Up Page for Students



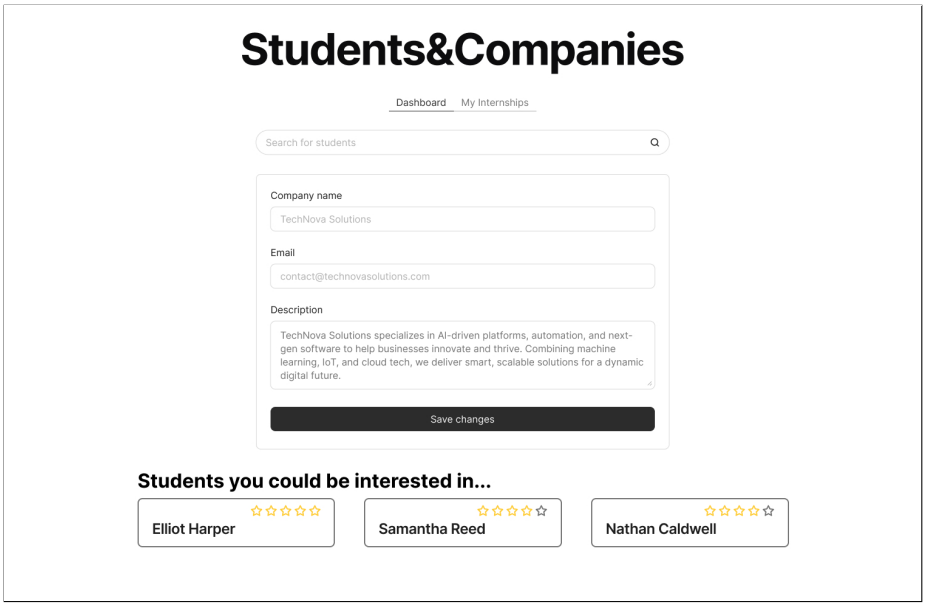
Sign Up Page for Companies

Dashboard

The ***Dashboard*** is the landing page of the website for every user. It shows the user's personal information, along with the recommendation based on the user's type, whether it is a student or a company.



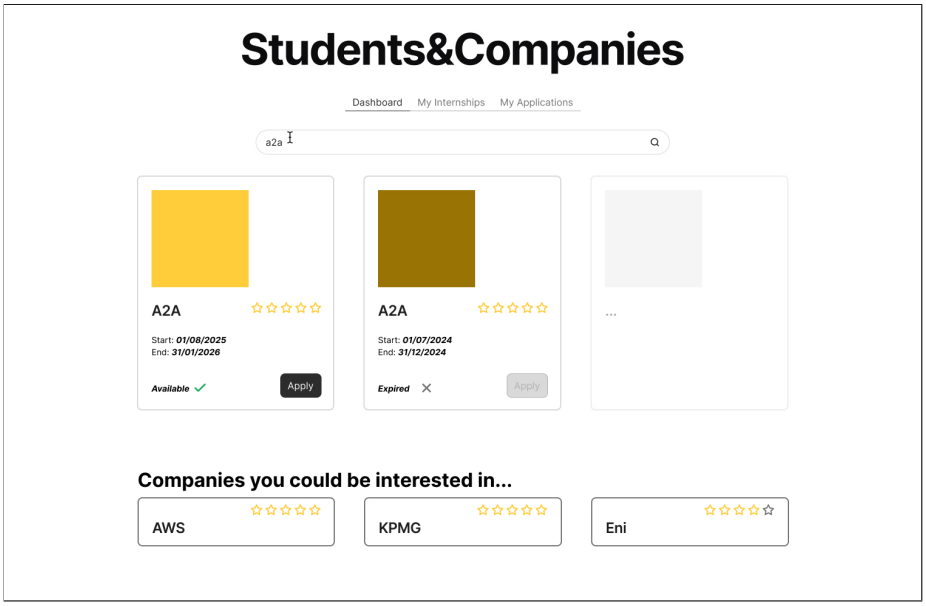
Dashboard for Students



Dashboard for Companies

Internship Search

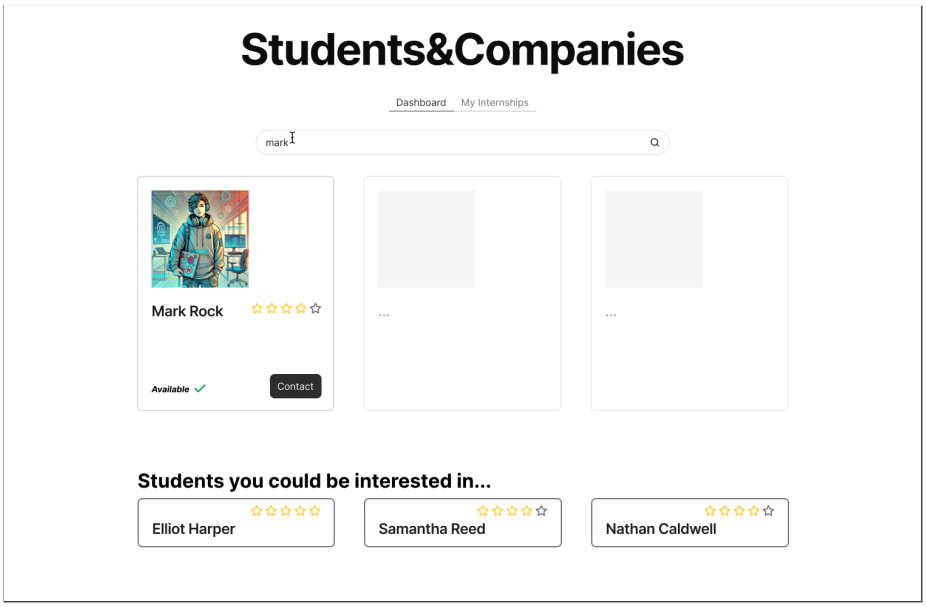
The ***Internship Search*** page allows students to search for internships based on the name and the keywords. The page shows a list of internships that match the search criteria. The student can apply for an internship by clicking on the *Apply* button.



Internship Search by Students

Student Search

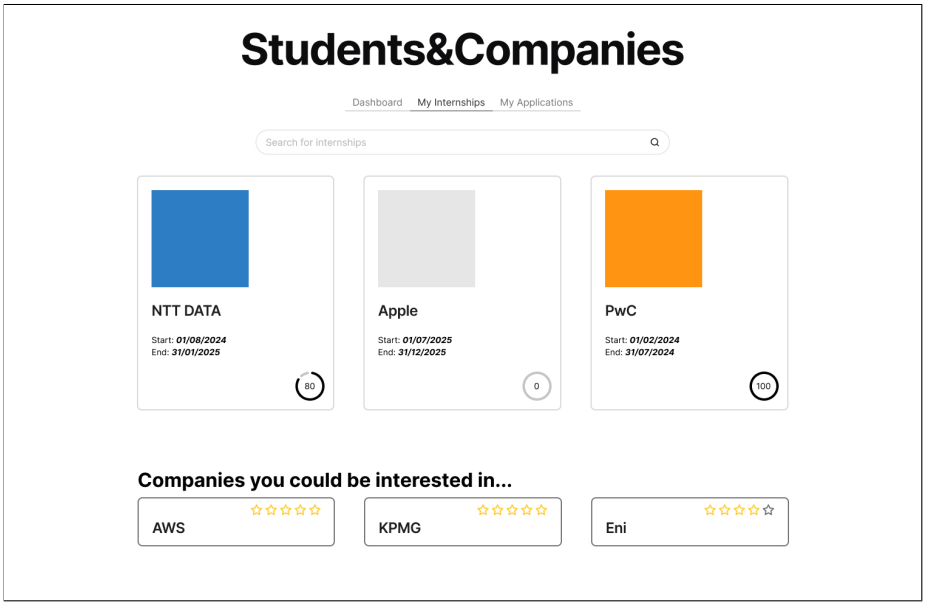
The ***Student Search*** page allows companies to search for students based on the name and the keywords. The page shows a list of students that match the search criteria. The company can select a student for an internship by clicking on the *Select* button.



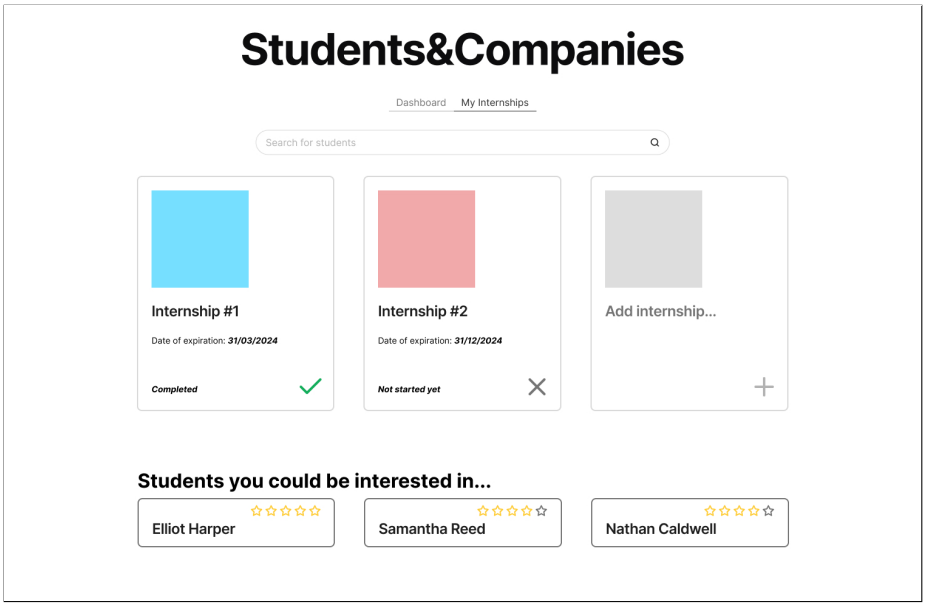
Student Search by Companies

My Internships Page

The ***My Internships Page*** shows the list of internships that the student has applied for. The page shows the status of the internships and possible recommendations based on the student's profile. In the same way, the ***My Internships Page*** for companies shows the list of internships that the company has created along with recommendations based on the company's needs.



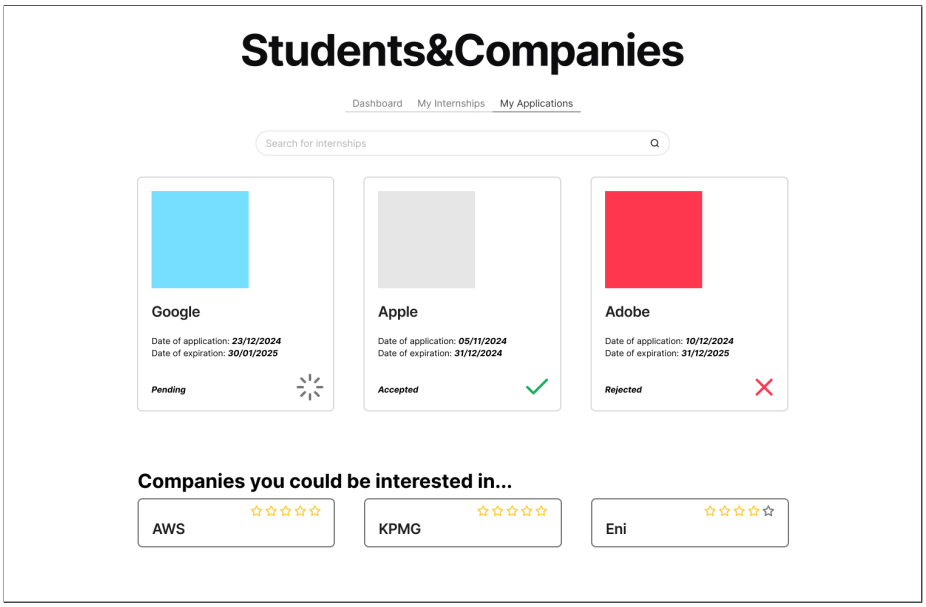
My Internships Page for Students



My Internships Page for Companies

My Applications Page

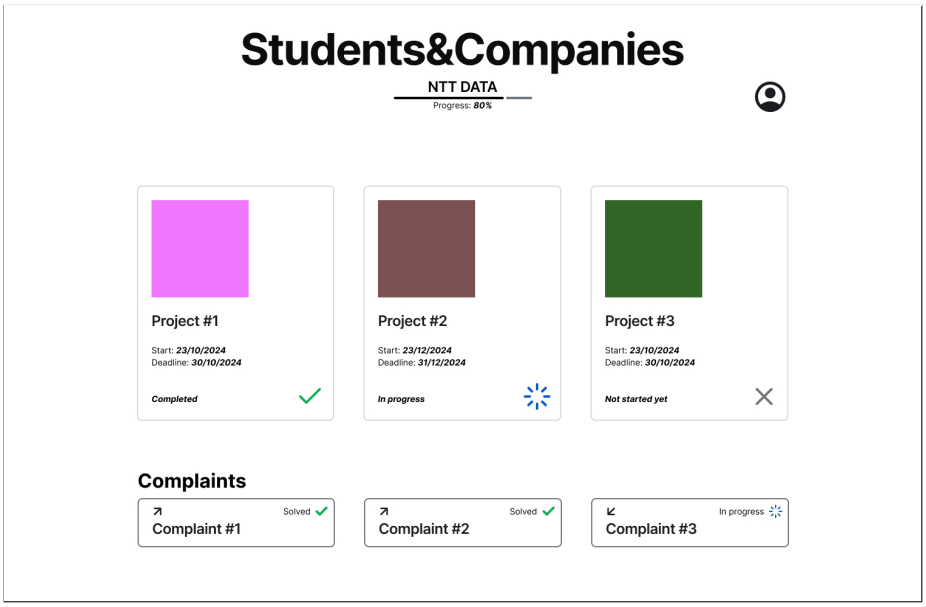
The ***My Applications Page*** shows the list of applications that the student has submitted. The page shows the status of the applications.



My Applications Page

Ongoing Internship Page

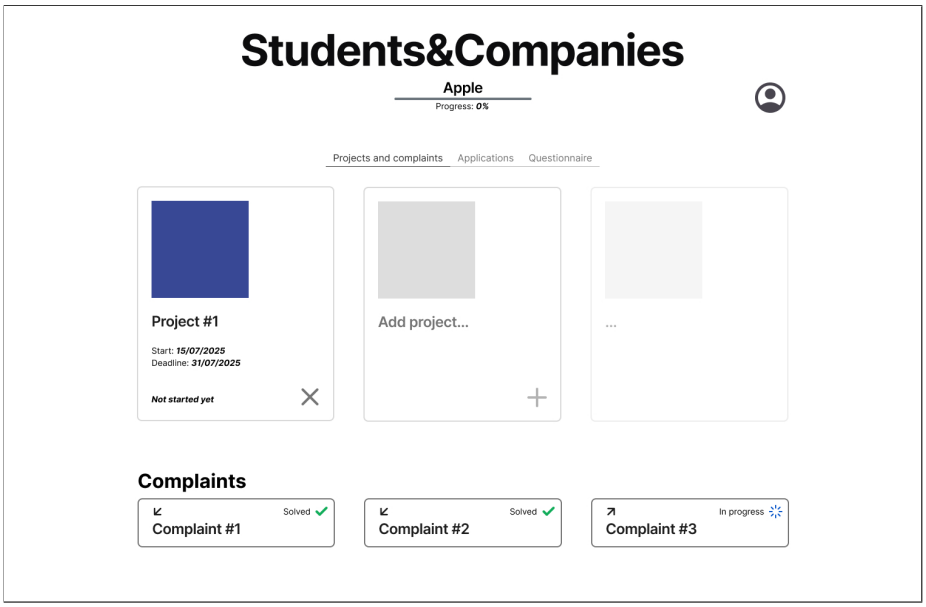
The ***Ongoing Internship Page*** shows data about an ongoing internship for the student. The page shows the progress of the internship, the projects related to the internship and the complaints that the student has submitted or received.



Ongoing Internship Page for Students

Internship Projects and Complaints Page

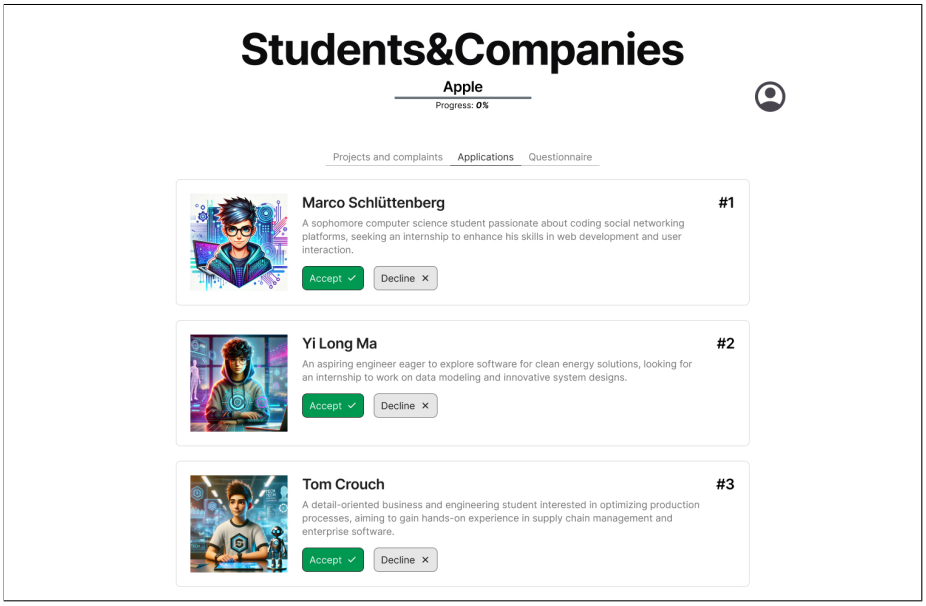
The ***Internship Projects Page*** shows the list of projects that the company has created for that internship. The page shows the status of the internships and the complaints that the company has submitted or received.



Internship Projects and Complaints Page for Companies

Selection Process Page

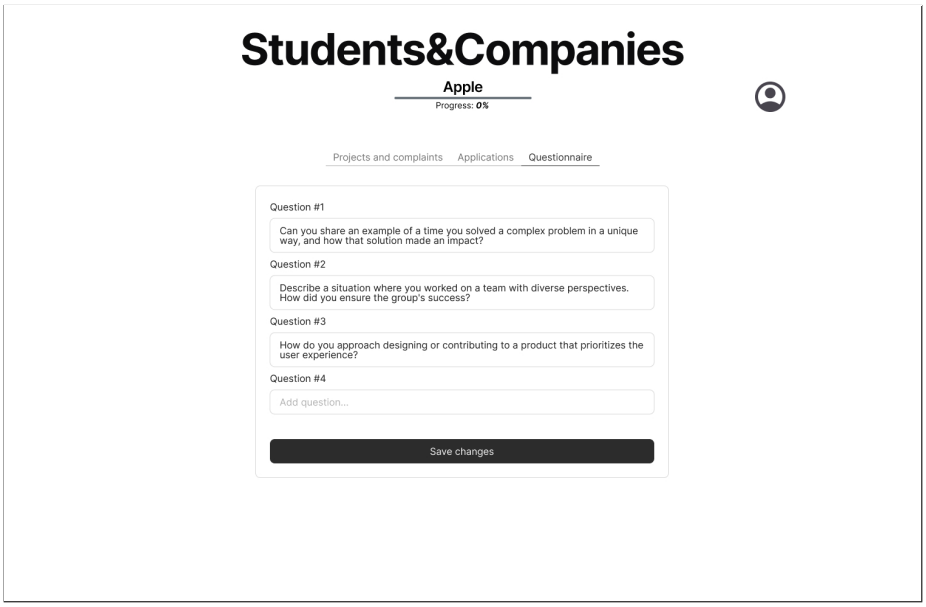
The ***Selection Process Page*** shows the list of students that have applied for an internship. The company can select the students for the internship by clicking on the *Accept* button or reject the students by clicking on the *Decline* button. Moreover, the students are sorted based on an overall score calculated by the system.



Students Selection Page for Companies

Questionnaire Publishing Page

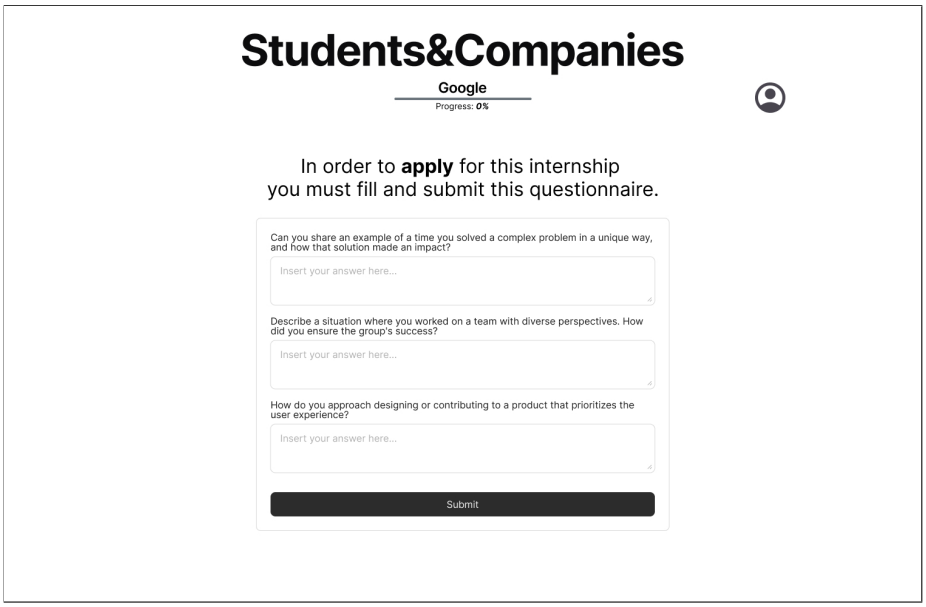
The ***Questionnaire Publishing Page*** allows companies to create and publish a questionnaire that is mandatory for applying to the internship. The page shows the list of questions that the company has created related to the questionnaire.



Internship Questionnaire Page for Companies

Questionnaire Compilation Page

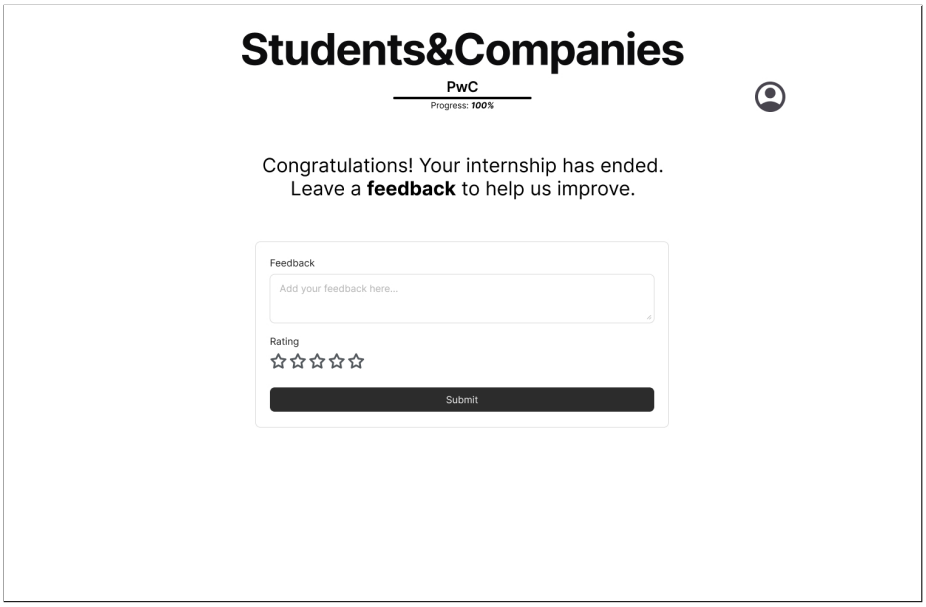
The ***Questionnaire Compilation Page*** allows students to compile the questionnaire proposed by the company. The page shows the list of questions that the company has created related to the questionnaire.



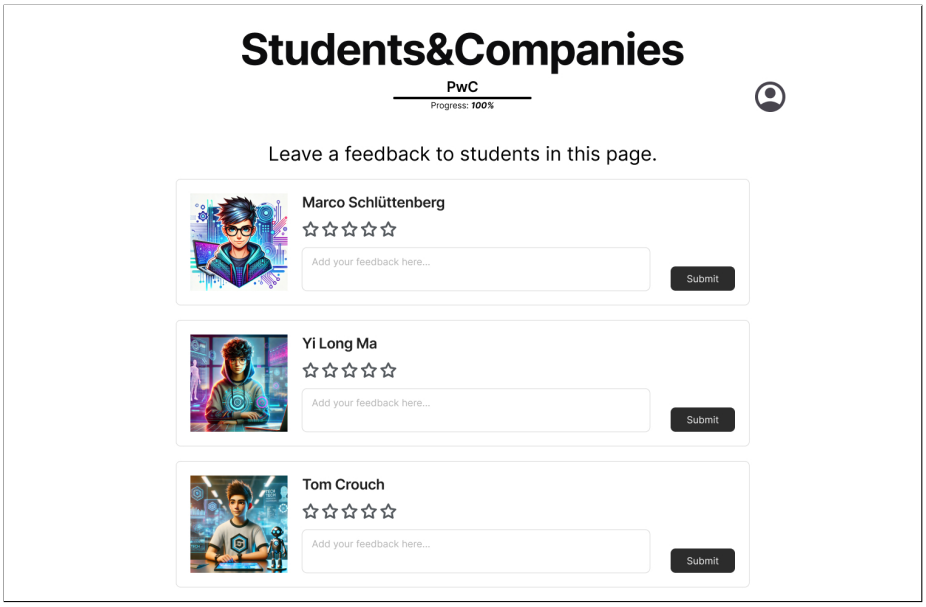
Internship Questionnaire Page for Students

Feedback Submission Page

The ***Feedback Submission Page*** allows students to submit feedback about an internship. The page offers a form where it is possible to rate the internship with up to five stars and to provide a comment about the experience. In the same exact way, the ***Feedback Submission Page*** for companies allows them to submit feedback about a student.



Feedback given by Students



Feedback given by Companies

4 Requirements Traceability

Requirements	**Components**
R1. The system allows students to sign up	AuthenticationService, DataManager
R2. The system allows companies HR to sign up.	AuthenticationService, NotificationSeervice, DataManager
R3. The system allows registered students to log in.	AuthenticationService, DataManager
R4. The system allows registered HRs of companies to log in.	AuthenticationService, DataManager
R5. The system allows students to insert their CV.	DashboardManager, DataManager
R6. The system allows users to insert and manipulate personal information.	DashboardManager, DataManager
R7. The system allows companies to create new available internships.	DashboardManager, InternshipManager, DataManager
R8. The system allows companies to set a deadline for available internships.	DashboardManager, InternshipManager, DataManager
R9. The system automatically prevents students from applying for expired internships.	DashboardManager, InternshipManager, DataManager
R10. The system allows students to apply for an available internship.	DashboardManager, InternshipManager, DataManager
R11. The system allows students to cancel an application to an internship.	DashboardManager, ApplicationManager, InternshipManager, DataManager
R12. The system allows students to proactively search for internships by using keywords.	DashboardManager, EvaluatorController, Preprocessor, DataManager

Requirements	Components
R13. The Recommendation system suggests available internships to students based on statistical analysis.	DashboardManager, EvaluatorController, Preprocessor, DataManager
R14. The Recommendation system recommends students to companies based on students' CVs.	DashboardManager, EvaluatorController, Preprocessor, DataManager
R15. The system allows companies to request students to apply for a given internship.	DashboardManager, InternshipManager, NotificationService,
R16. The system allows students to accept or deny requests for internships received from companies.	DashboardManager, InternshipManager, ApplicationManager, NotificationService, DataManager
R17. The system allows companies to publish structured questionnaires on the internship page.	DashboardManager, InternshipManager, DataManager
R18. The system allows students to compile the questionnaires proposed by the company.	DashboardManager, InternshipManager, DataManager
R19. The system ranks students who applied to the internship based on the results of the questionnaires.	DashboardManager, InternshipManager, EvaluatorController, Preprocessor, ApplicationManager, DataManager
R20. The system allows companies to choose for a maximum number of students to be accepted.	DashboardManager, InternshipManager, DataManager
R21. The system allows both students and companies to provide feedback about each other and to suggest potential improvements after the internship is over.	DashboardManager, InternshipManager, DataManager
R22. The system notifies students whether their application for an internship has been accepted or declined.	DashboardManager, InternshipManager, ApplicationManager, NotificationService, DataManager

Requirements	**Components**
R23. The system allows students and companies to complain about problems during internships.	DashboardManager, InternshipManager, DataManager
R24. The system allows companies to check ratings of students applied to one internship.	DashboardManager, InternshipManager, EvaluatorController, Preprocessor, DataManager
R25. The system allows Universities to log in.	AuthenticationService, DataManager
R26. The system allows Universities to monitor the internships by their students.	DashboardManager, InternshipManager, DataManager
R27. The system allows Students and Companies to send email to each other.	DashboardManager, NotificationService

5 Implementation, Integration and Test Plan

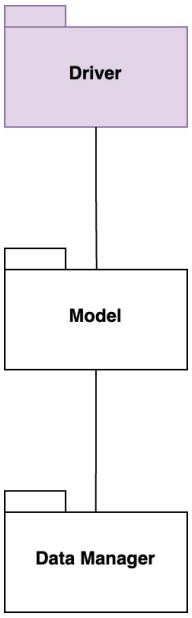
The system will be implemented and tested through a *bottom-up* approach and the same strategy will be used for both server, recommendation server and client. Since some interfaces like DAOs are assumed to be reliable, they won't need to be tested.

5.1 Overview

In this section we will delve into the actual testing of every piece of our system, from server-side to recommendation service, and finally the client-side.

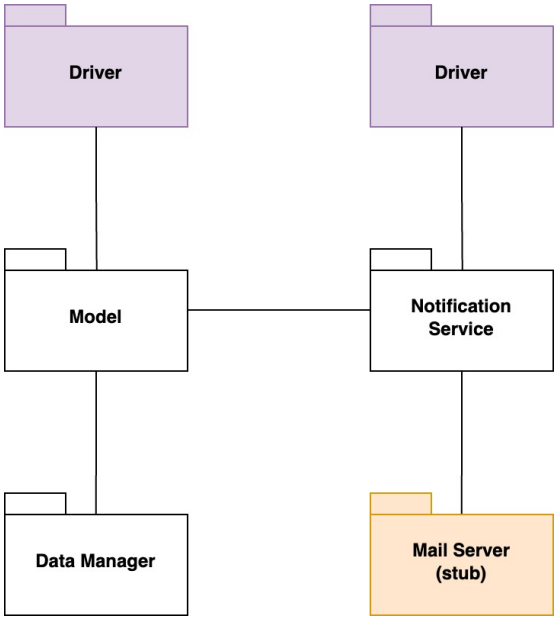
Server Testing

In the first step of server testing, we will implement the *Data Manager* and our *Model* (under the assumption of an MVC pattern implementation). Through the *Model* it's possible to access the actual database by means of the *Data Manager*. Our *Model* will be tested using a *Driver* that will substitute the not yet implemented components.



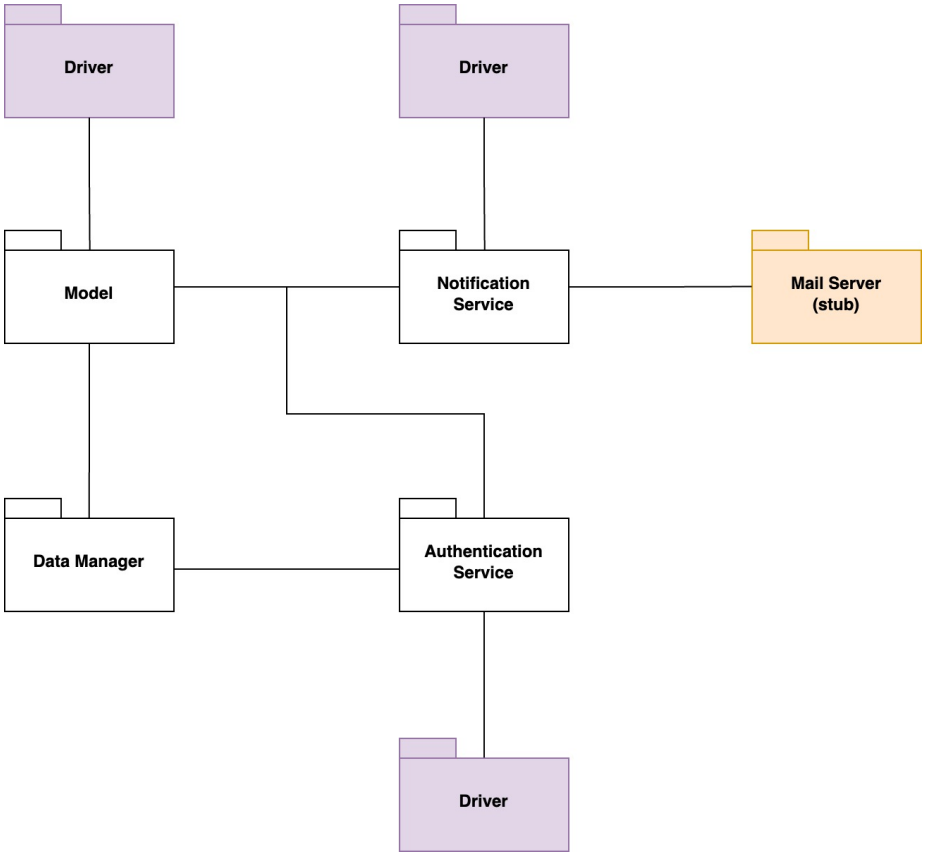
Server Testing - Step 1

In the second step, the *Notification Service* will be implemented and tested using *Driver* that will substitute the *Internship Manager*; moreover a *Stub* will be used to emulate a *Mail Server*.



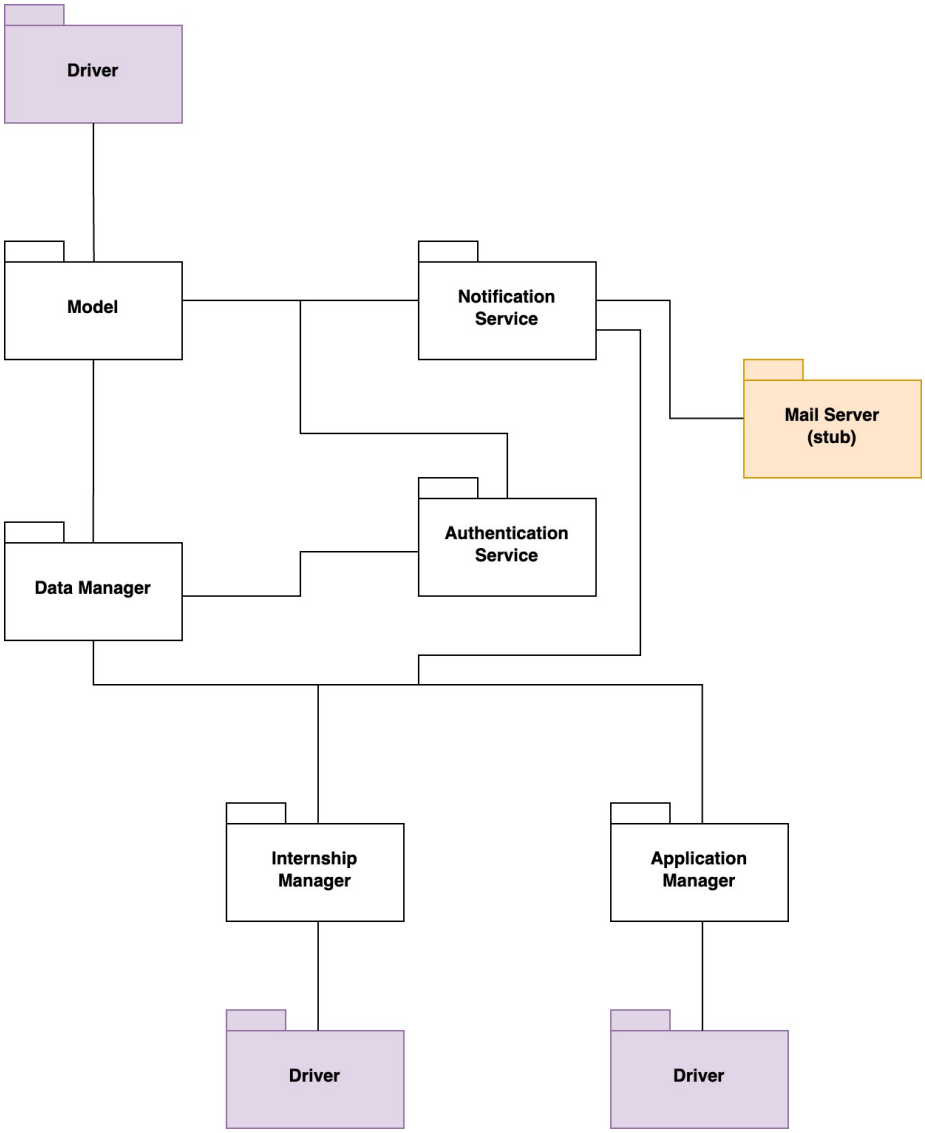
Server Testing - Step 2

In the third step, we will implement the *Authentication Service* that will be tested by means of a *Driver* substituting the *Dashboard Manager*.



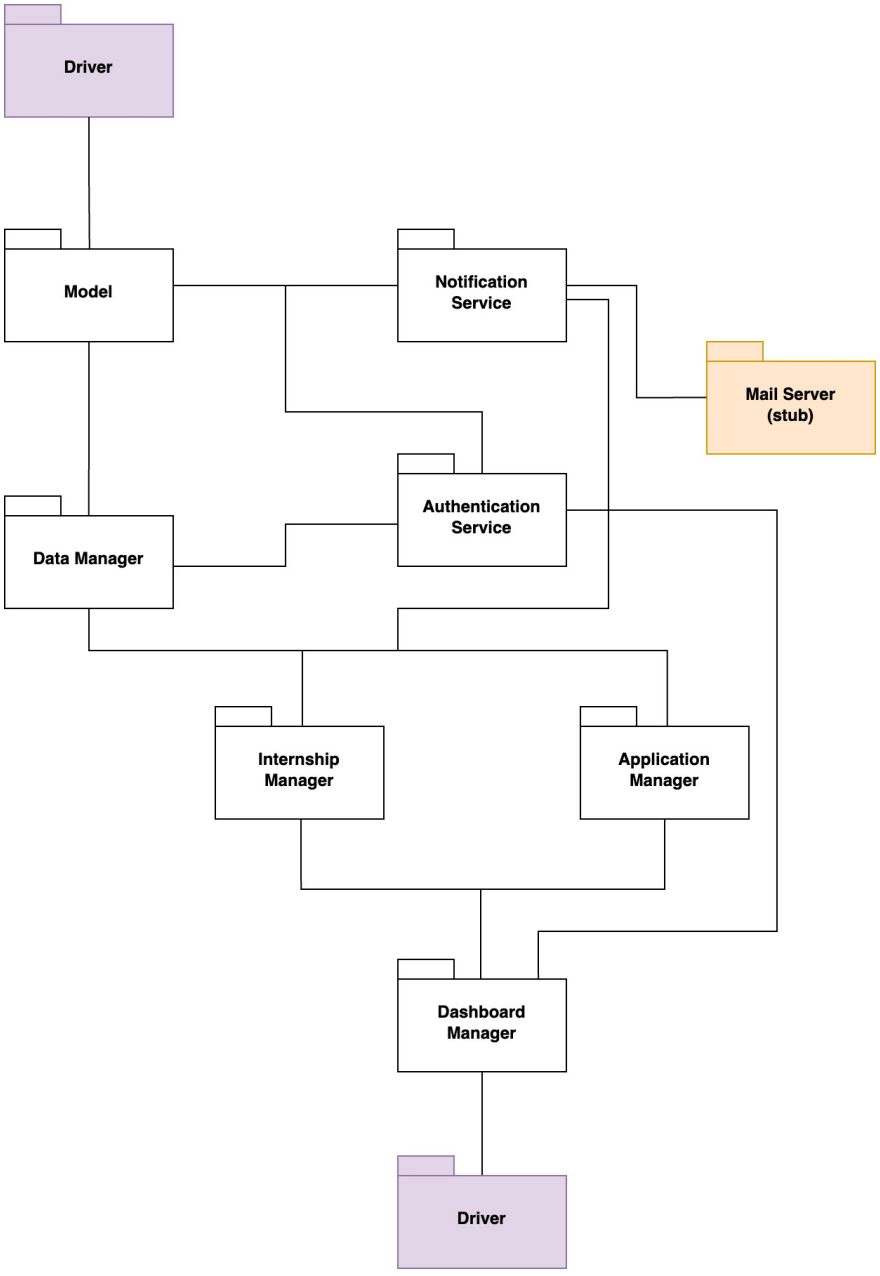
Server Testing - Step 3

In the fourth step, both the *Internship Manager* and the *Application Manager* will be implemented and a *Driver* each will substitute the *Dashboard Manager*.



Server Testing - Step 4

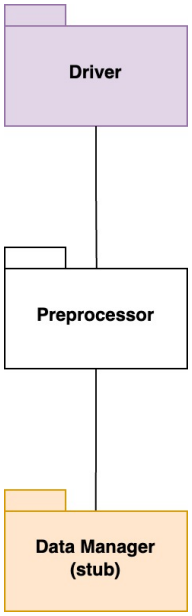
Finally, in the fifth step, the *Dashboard Manager* will be implemented and a *Driver* will substitute the *Client* layer.



Server Testing - Step 5

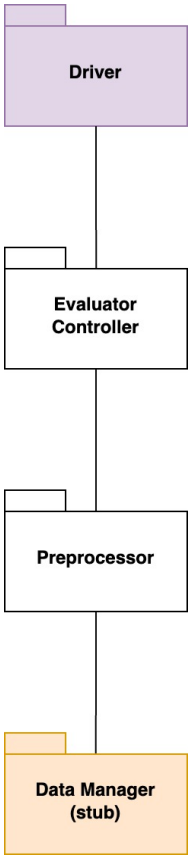
Recommendation Server Testing

In the first step, we will implement the *Preprocessor* that will be tested by using a *Driver* that substitutes the yet not implemented components. Moreover, a *Stub* will be used to emulate the *Data Manager* from the *Server*.



Recommendation Server Testing - Step 1

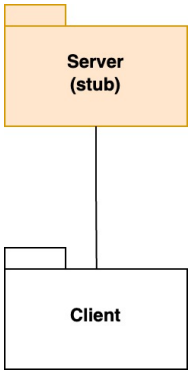
In the second step, the *Evaluator Controller* will be implemented and tested by means of a *Driver* that substitutes the *Dashboard Manager* from the *Server*.



Recommendation Server Testing - Step 2

Client Testing

After implementing and testing all the other components, we can finally implement and test our *Client* by means of a *Stub* that emulates the back-end of our system.



Client Testing - Step 1

6 Effort Spent

Denis Sanduleanu	
Chapter	**Effort (in hours)**
1	0
2	10
3	0
4	2
5	0

Francesco Gangi	
Chapter	**Effort (in hours)**
1	1
2	9
3	11.5
4	0
5	4

7 References

- Diagrams made with: **Diagrams.net**
- Mock-ups made with: **Figma**